

Compresión JPEG

Servicios multimedia



Grupo de Tecnología Electrónica
y Comunicaciones



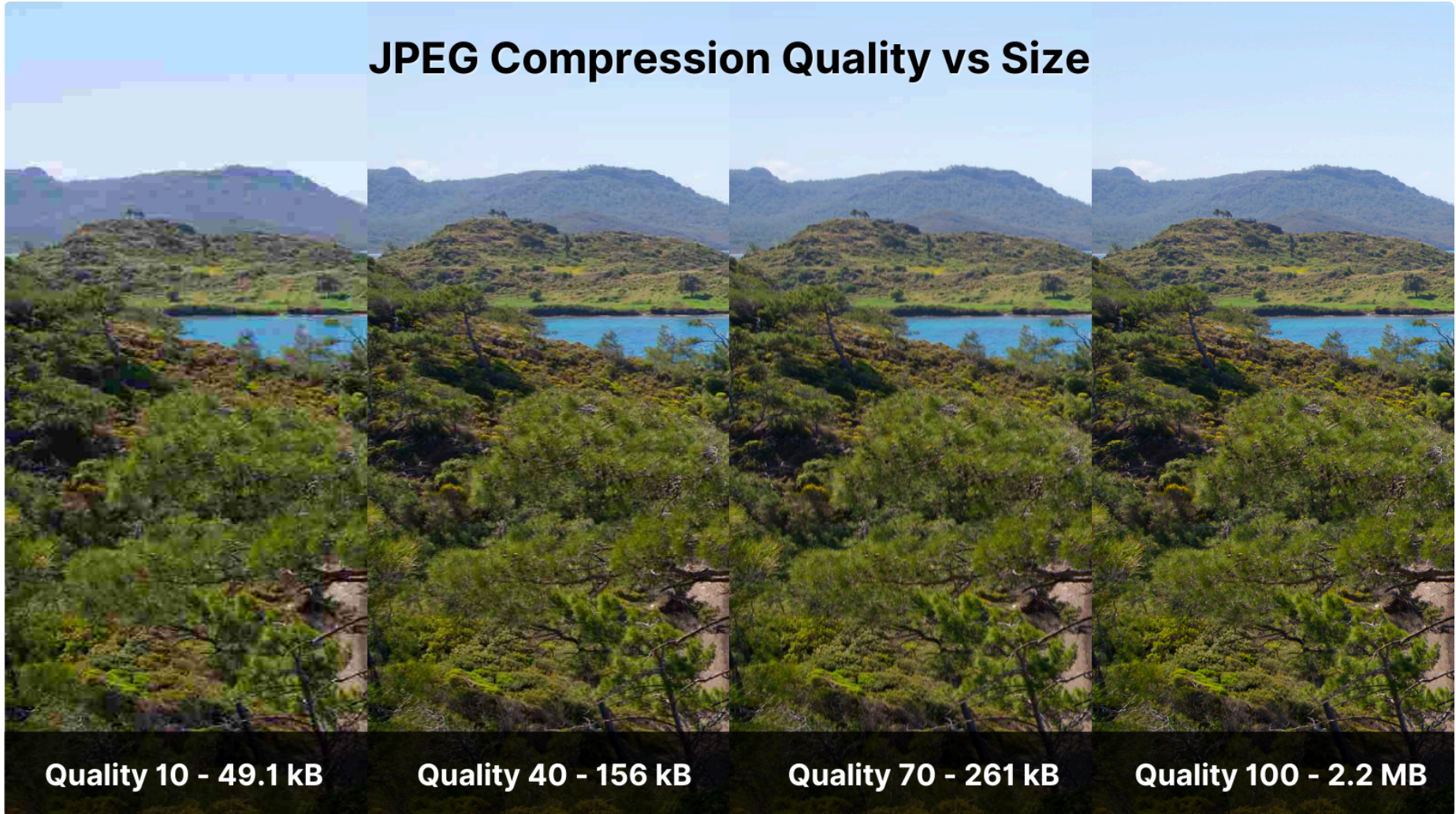
UNIVERSIDADE DA CORUÑA

UNIVERSIDADE DA CORUÑA

Historia

¿Para qué JPEG?

JPEG Compression Quality vs Size



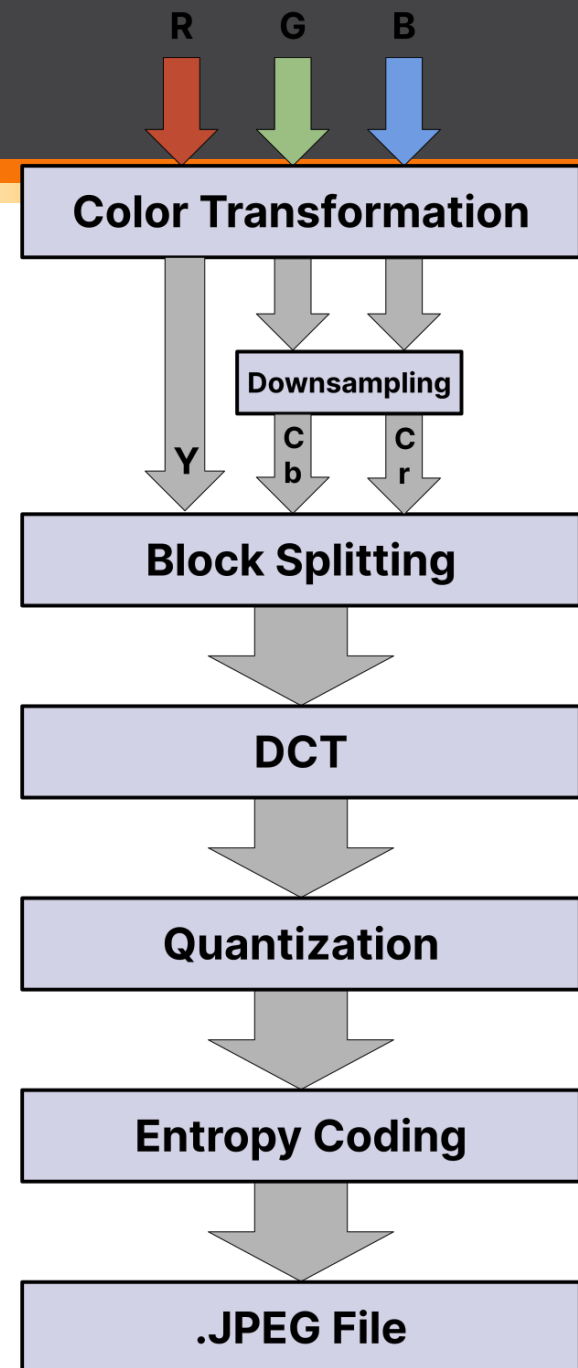
- JPEG significa Joint Photographic Experts Group.
 - Comité de expertos promovido por ISO y CCITT
 - Formado en 1986 para desarrollar un estándar para compresión de imágenes
 - 1987: ISO TC 97 se convierte en ISO/IEC JTC1
 - 1992: CCITT se convierte en ITU-T
- El primer estándar fue lanzado en 1992, aprobado en septiembre de 1992 como ITU-T Recommendation T.81 y en 1994 como ISO/IEC 10918-1.
- Actualmente, JPEG es uno de los dos sub-grupos del JTC1 ISO/IEC JTC 1/SC29/WG 1 “Coding Still Pictures”.
- En la ITU-T, el grupo que se encarga es el ITU-T SG16.

- JPEG está orientado a la compresión de imágenes digitales:
 - Fotografía, retratos, cuadros de escenas “realistas”, etc.
 - Asume variaciones suaves en el tono y en el color.
- JPEG no es adecuado para dibujos lineales, texto, etc. en donde se producen contrastes abruptos de tono y/o color.
 - Para dibujos lineales mejor usar esquemas de compresión sin pérdidas como PNG, TIFF ó incluso GIF.
- JPEG no es adecuado para imágenes que sufrirán múltiples ediciones.
 - Cada vez que se edita y guarda la imagen se pierde calidad (digital regeneration loss)

- La compresión en JPEG se apoya en dos ideas:
 - Eliminar información prescindible en una imagen:
 - JPEG representará de forma menos precisa la información de más alta frecuencia.
 - Menor sensibilidad cuando hay transiciones abruptas de luminosidad y color
 - Se asume que las transiciones son en general suaves
 - Explotar la redundancia espacial presente en una imagen:
 - Se elimina la redundancia con técnicas de codificación de fuente (e.g. codificación entrópica) y suele ser sin pérdidas.



- Buscamos bajar el tamaño de la imagen sin bajar mucho la calidad percibida.
- Para ello intentamos eliminar la información a la que el ojo humano es menos sensible.
- El proceso para convertir a JPEG consiste de varias fases.

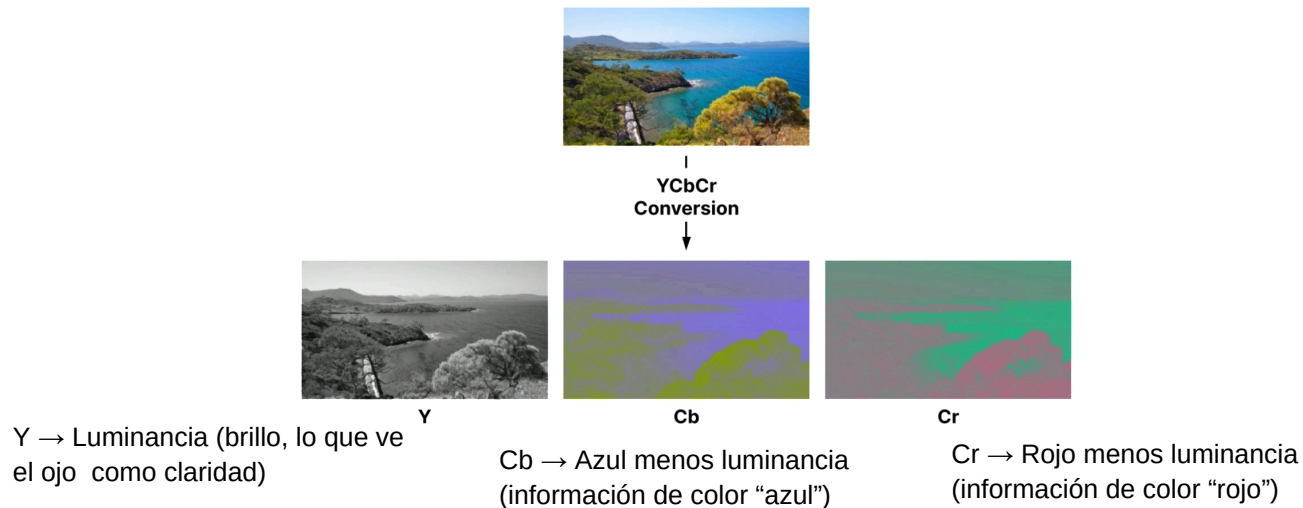


Funcionamiento

Transformación de color

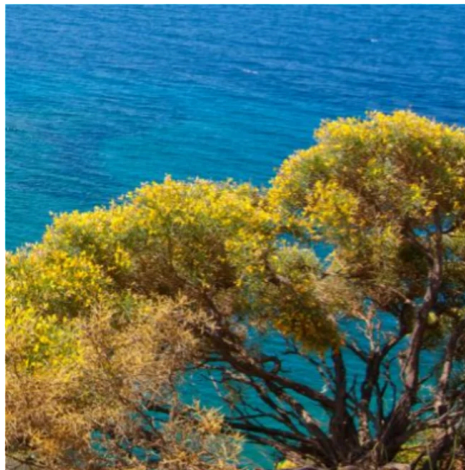
Red, Green, Blue -> Es el formato original de la mayoría de imágenes en un PC

- En la primera fase, se pasa la imagen de RGB a un modelo de color distinto: YCbCr.
- La información tras esta fase es la misma, no se pierde nada.
- ¿Para qué hacemos esto? El ojo humano es más sensible a los cambios de intensidad que a los de color, así que separando la componente de luminancia (Y) de las crominancias (Cb, Cr) podemos operar sobre cada componente por separado.



Downsampling de crominancias

- En la segunda fase, podemos reducir el tamaño de las imágenes de crominancia haciendo un downsampling. Es hacer más pequeña la parte del color de la imagen.
- Como el ojo humano es menos sensible a este tipo de información, la imagen apenas pierde calidad perceptual.
- Esta fase sí elimina información de la imagen original. Sí, se pierde un poco de información del color, porque estás usando menos detalle.



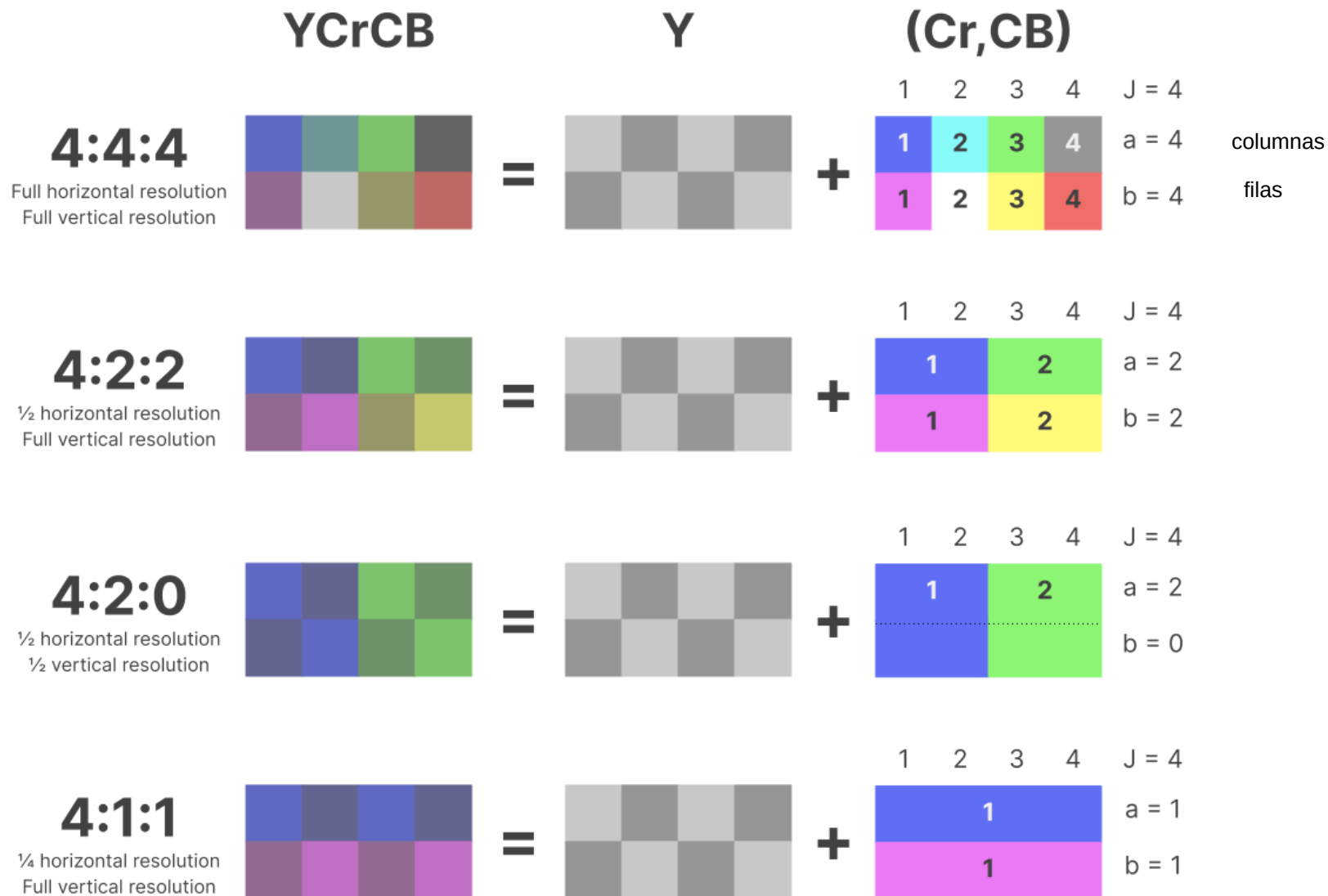
**JPEG Compression
No Downsampling**
323 kb



**JPEG Compression
4:2:0 Downsampling**
176 kb

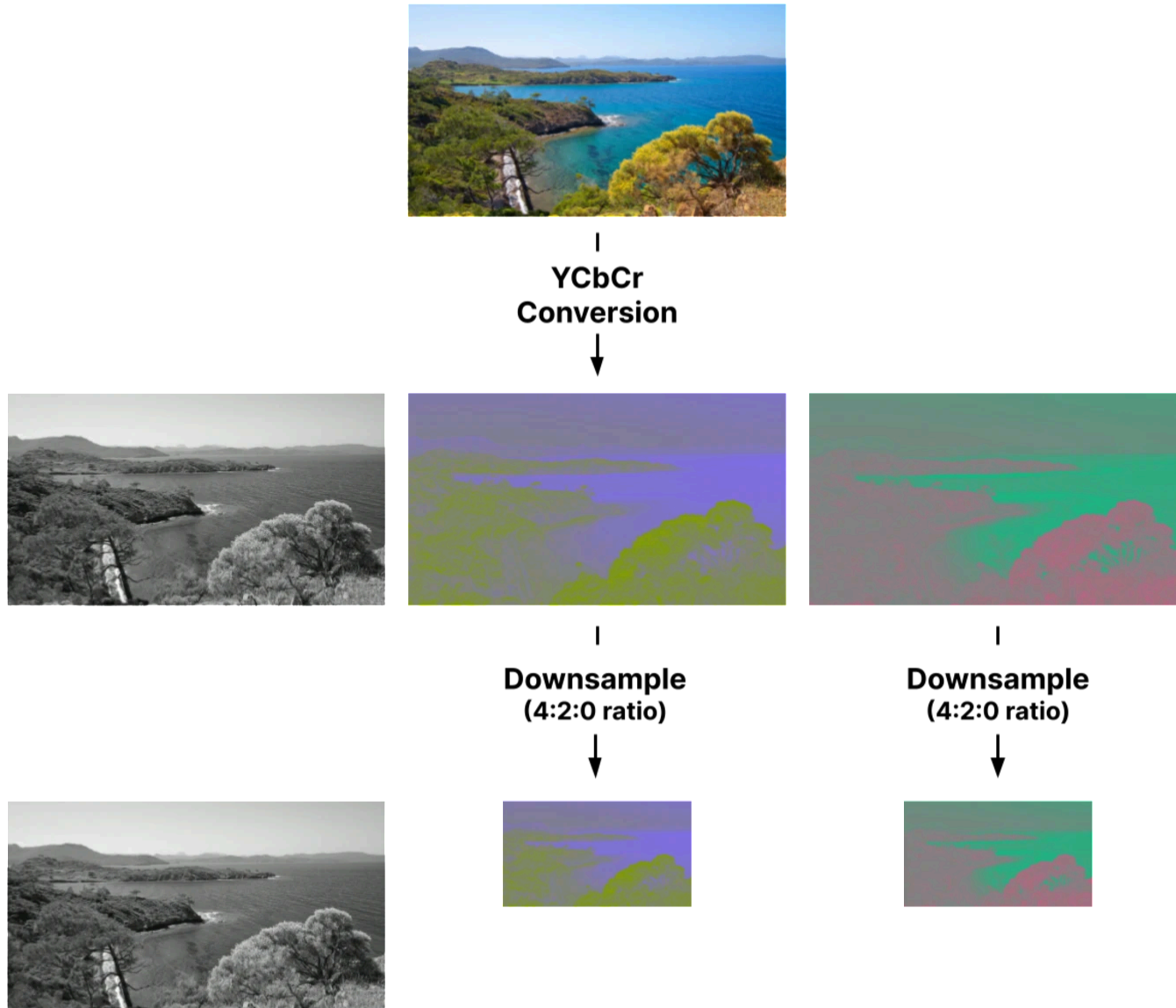
Funcionamiento

Downsampling de crominancias



Funcionamiento

Downsampling de crominancias



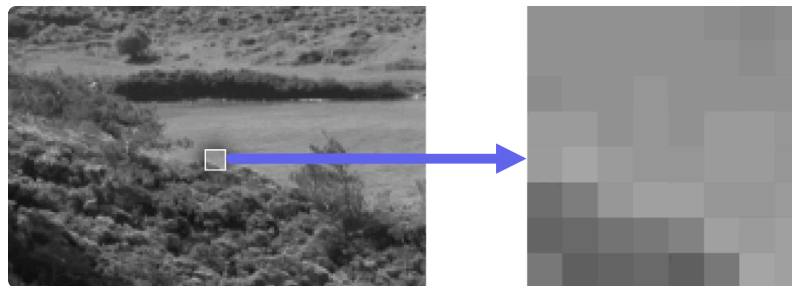
Funcionamiento

División en bloques de 8x8

- Para las siguientes fases, la imagen se procesa en bloques de 8x8px.
- Si la imagen no es múltiplo de 8x8, se fuerza añadiendo pixels con valor 0, repitiendo los del borde o interpolando.
 - Ejemplo: una imagen de 160 x 240 píxeles daría lugar a
 - $160 \times 240 = 8 \times 20 \times 8 \times 30$
 - $20 \times 30 = 600$ bloques
- Esto se hace para las tres componentes YCbCr.

Se divide en bloques.

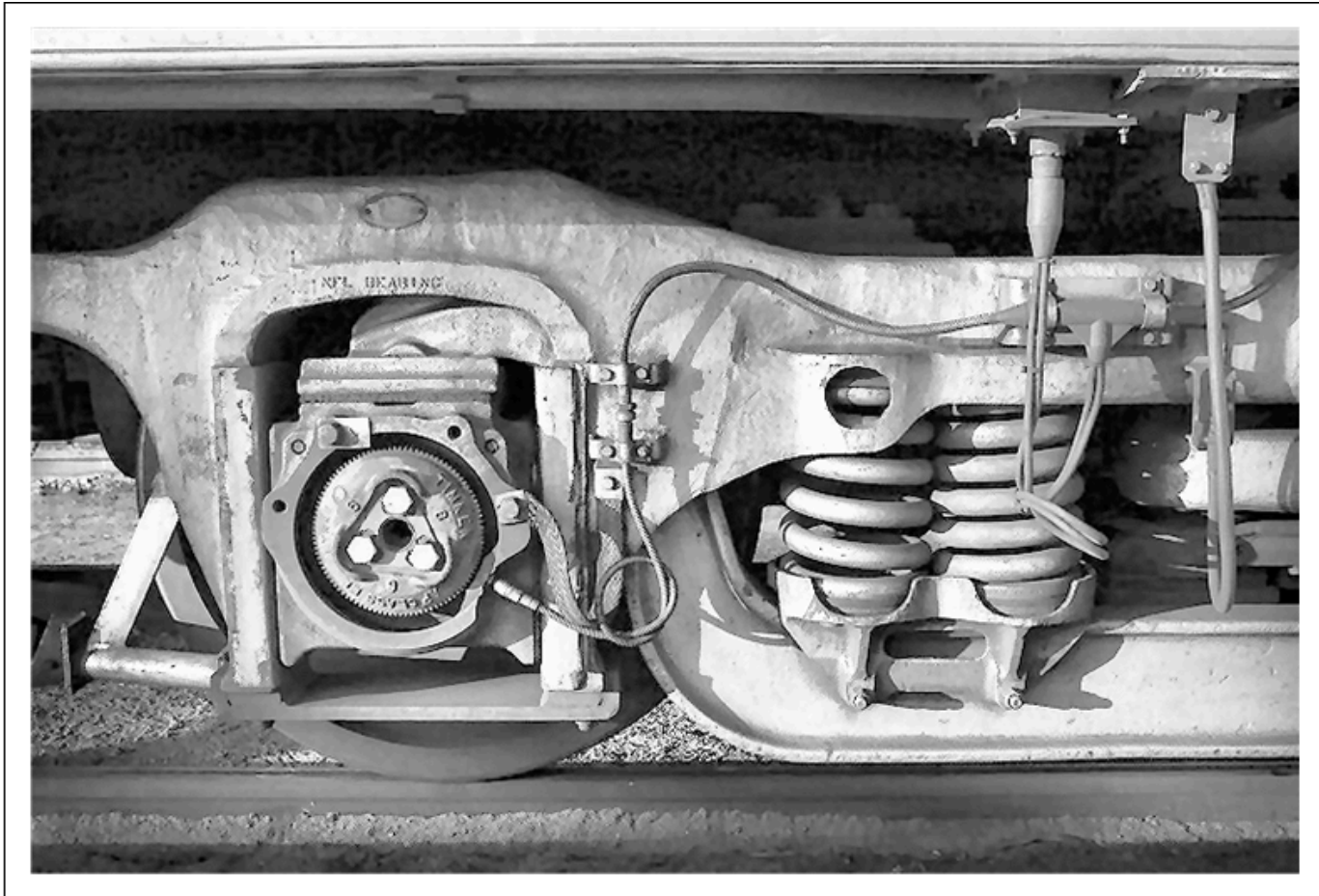
Cada uno de estos bloques tiene 64 píxeles



Funcionamiento

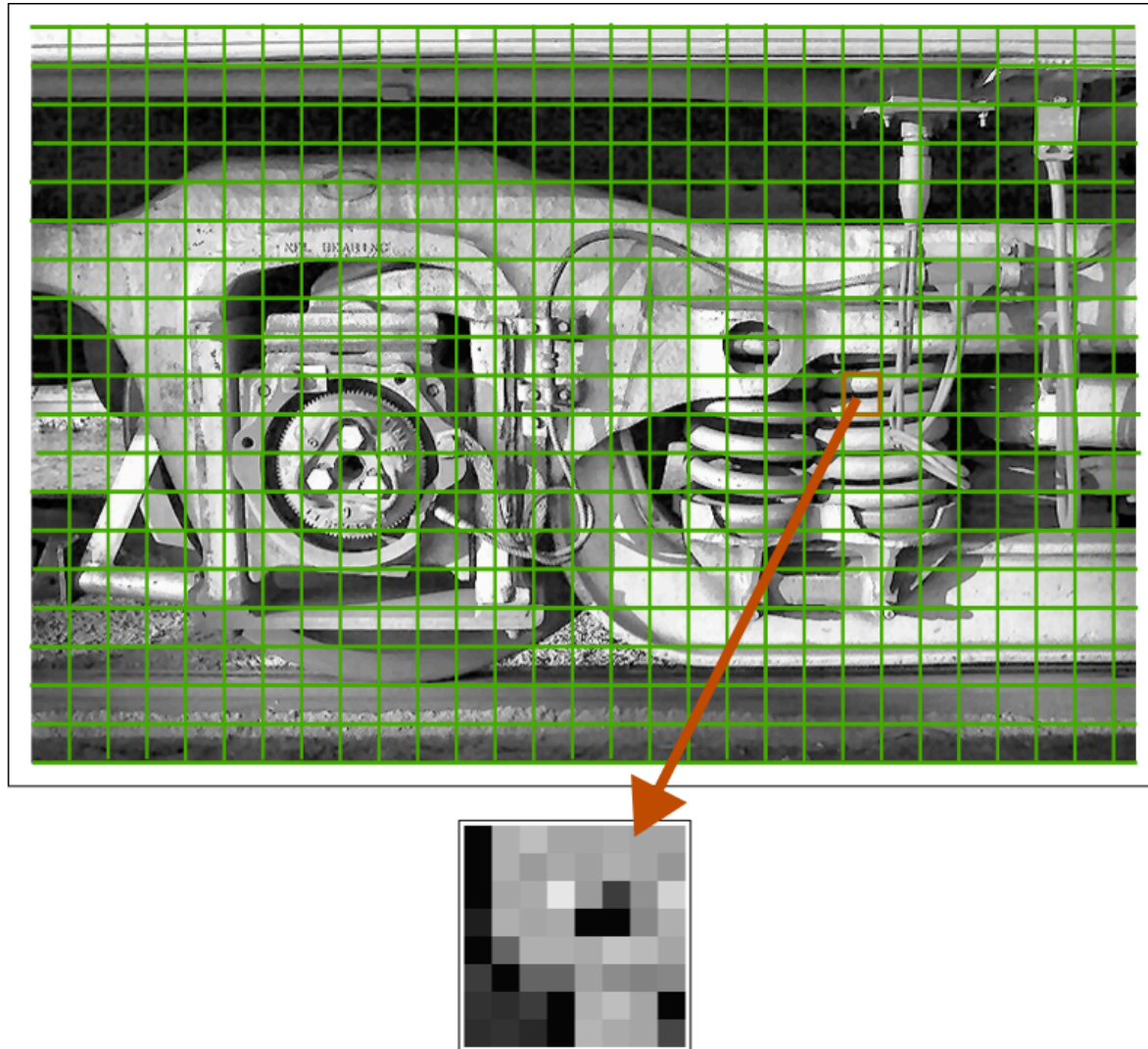
División en bloques de 8x8

- Ejemplo: partimos de la siguiente foto (componente de luminancia):



Funcionamiento

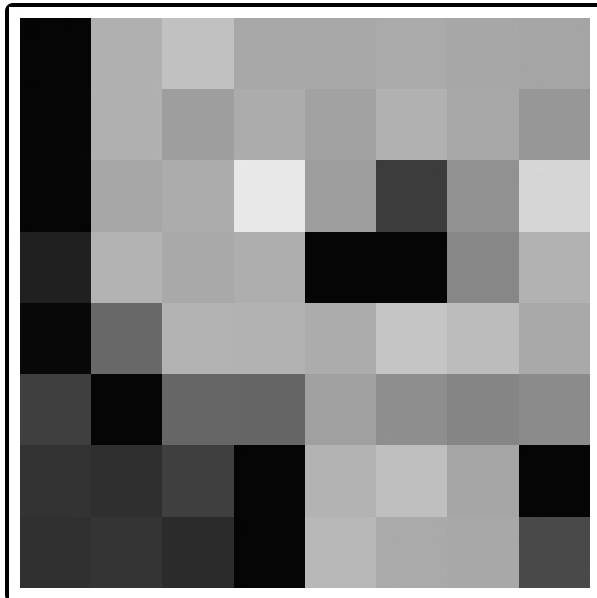
División en bloques de 8x8



Funcionamiento

División en bloques de 8x8

- Cada bloque se corresponde con una matriz de 64 elementos con los valores de luminancia para los píxeles de ese bloque:

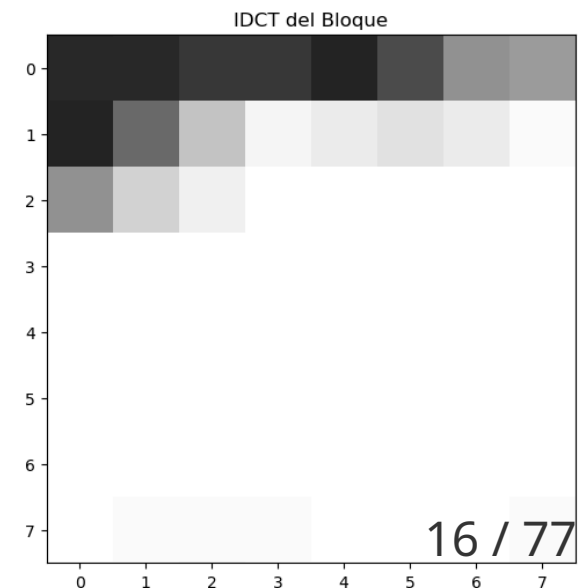
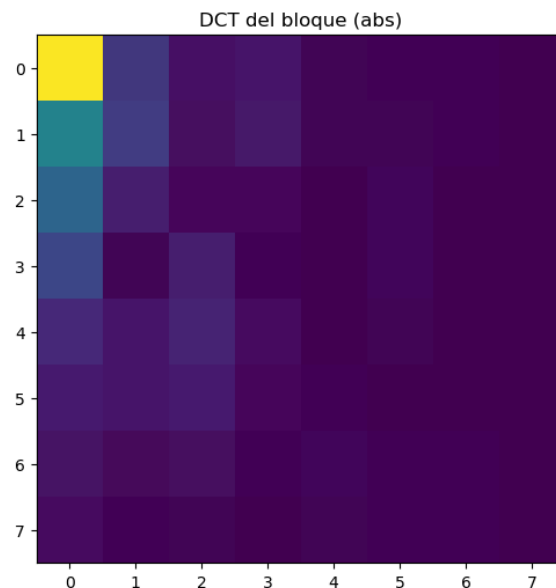
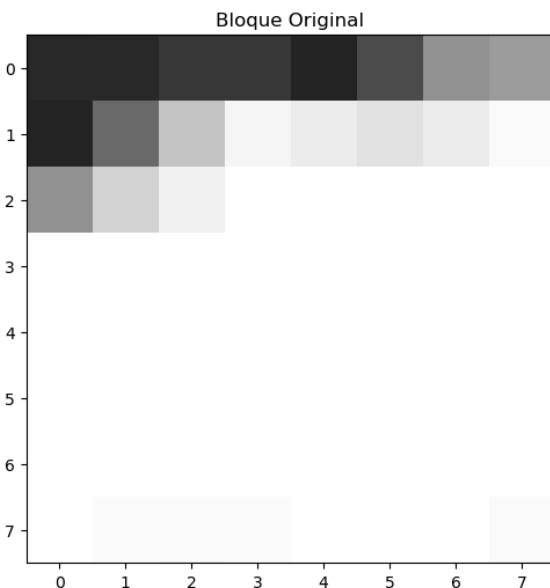


5	176	193	168	168	170	167	165
6	176	158	172	162	177	168	151
5	167	172	232	158	61	145	214
33	179	169	174	5	5	135	178
8	104	180	178	172	197	188	169
63	5	102	101	160	142	133	139
51	47	63	5	180	191	165	5
49	53	43	5	184	170	168	74

- El ojo humano detecta mejor cambios suaves de intensidad (frecuencias bajas), que los cambios bruscos (frecuencias altas).
- Una buena idea para comprimir sería entonces eliminar la información de las frecuencias altas y mantener la de las bajas.
- Pero solo viendo los píxeles, no podemos saber qué parte de la información es la que se corresponde a las frecuencias bajas y cuál a las altas.



- La DCT (Discrete Cosine Transform) es una transformada que nos permite ver la imagen en el dominio de la frecuencia.
- La DCT es una transformada discreta, similar a la de Fourier, pero basada únicamente en funciones coseno.
- No elimina información, simplemente la representa en otro dominio.



- Las transformadas se basan en representar una señal como una combinación lineal de funciones base ortogonales:

$$x(n) = \sum_{k=1}^K a_k g_k(n) \quad \forall N$$

donde:

- $g_k(n)$: Funciones base
- N : Número de elementos en el vector de entrada
- K : Número de funciones de la base

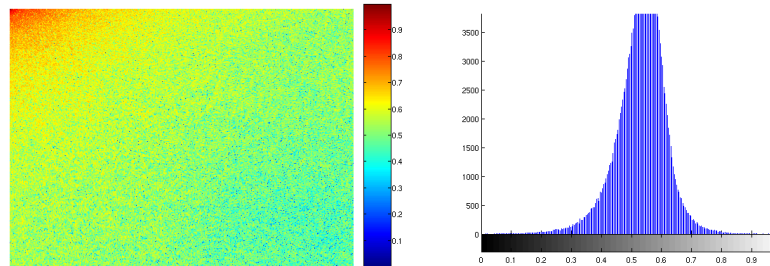
- Con la DCT podemos ver la imagen en el dominio de la frecuencia y eliminar parte de la información de las frecuencias altas.
- Tras esa operación, se pasa de nuevo al dominio original (los píxeles).
- En términos de MSE, la transformada óptima sería la Karhunen–Loève (KL).
 - Pero es muy compleja de calcular y requiere calcular la matriz de covarianza de cada bloque.
 - Para altas correlaciones entre píxeles, la DCT es una buena aproximación.
 - Además necesita menos coeficientes que la DFT para representar un conjunto de datos.

Funcionamiento

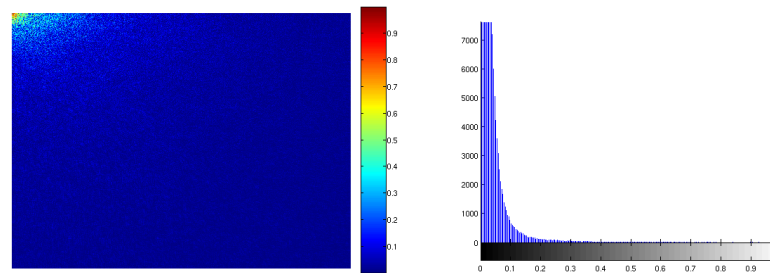
La DCT



DFT



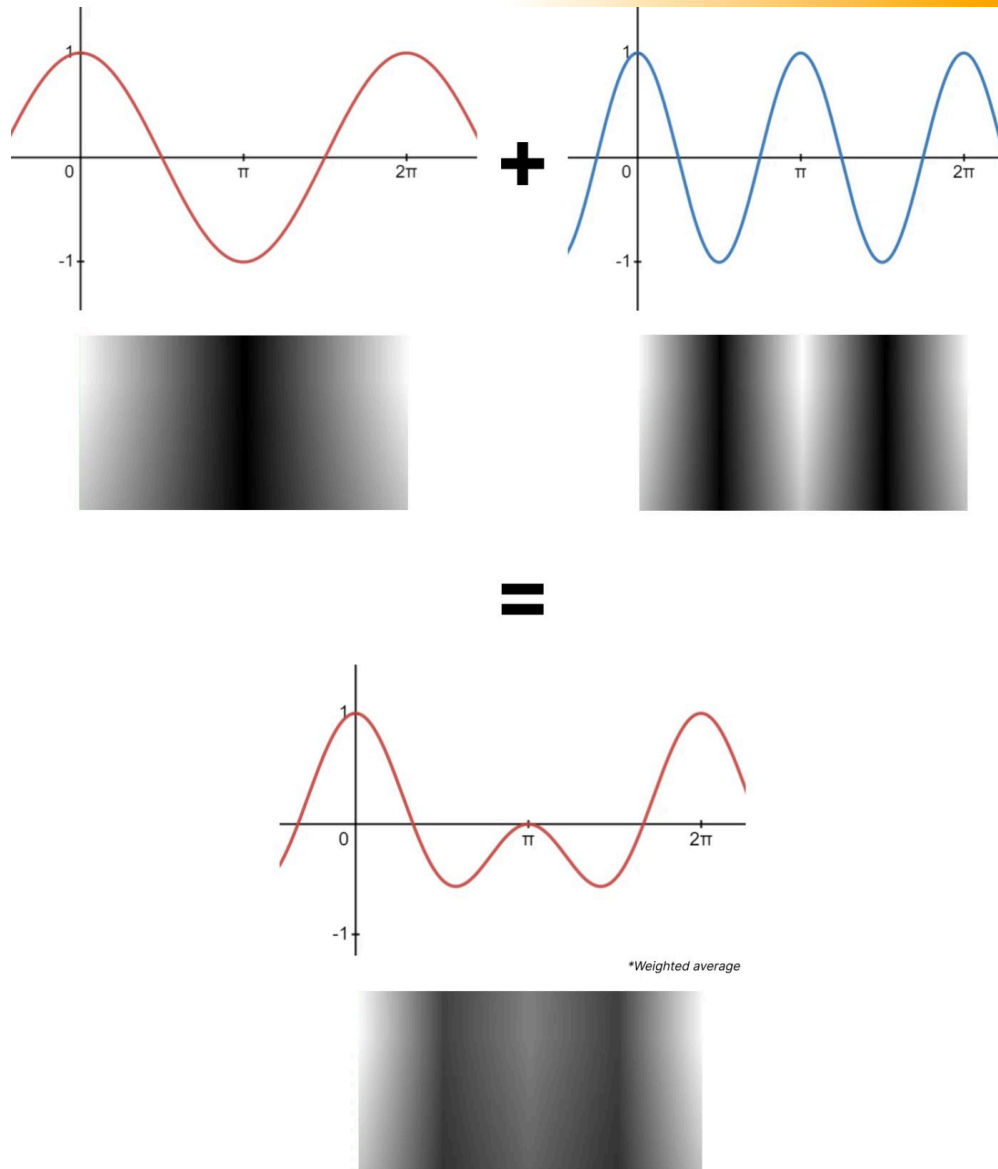
DCT



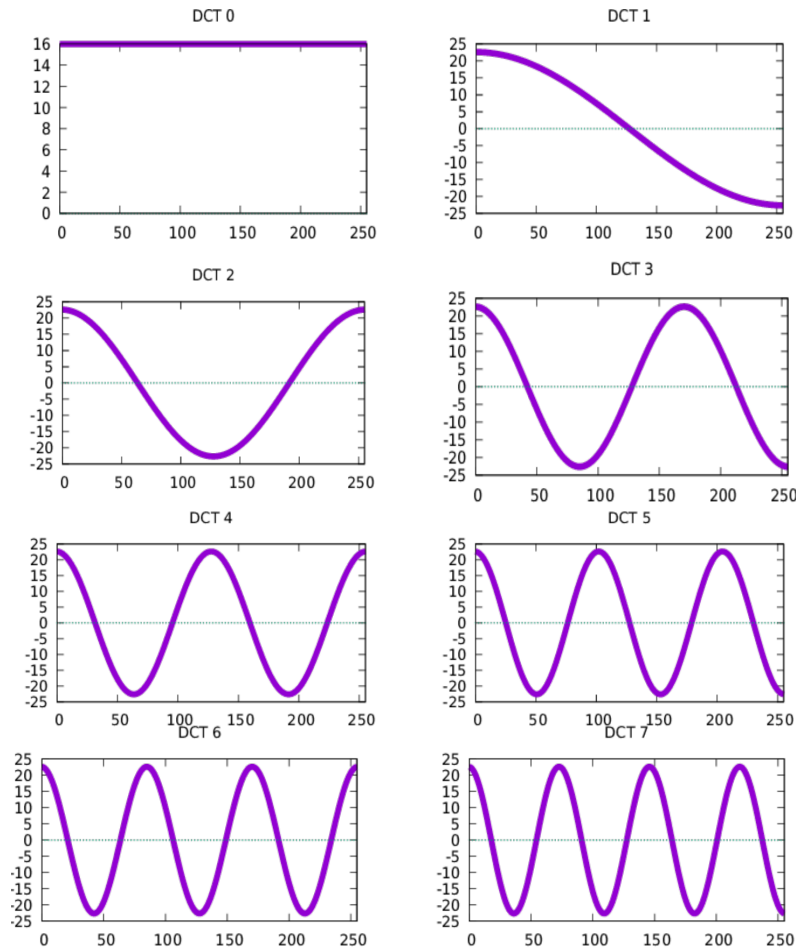
- Significado físico de la DCT:
 - Transiciones suaves en la entrada se representan con las bases correspondientes a los cosenos de menor frecuencia
 - Transiciones abruptas en la entrada son capturadas con las bases correspondientes a los cosenos de mayor frecuencia
- El coeficiente correspondiente a la frecuencia 0 se corresponde con la media de la señal de entrada
 - Se denomina coeficiente DC
 - El resto de coeficientes se denominan AC
- La DCT no comprime ni elimina información; simplemente representa la señal en otro dominio

Funcionamiento

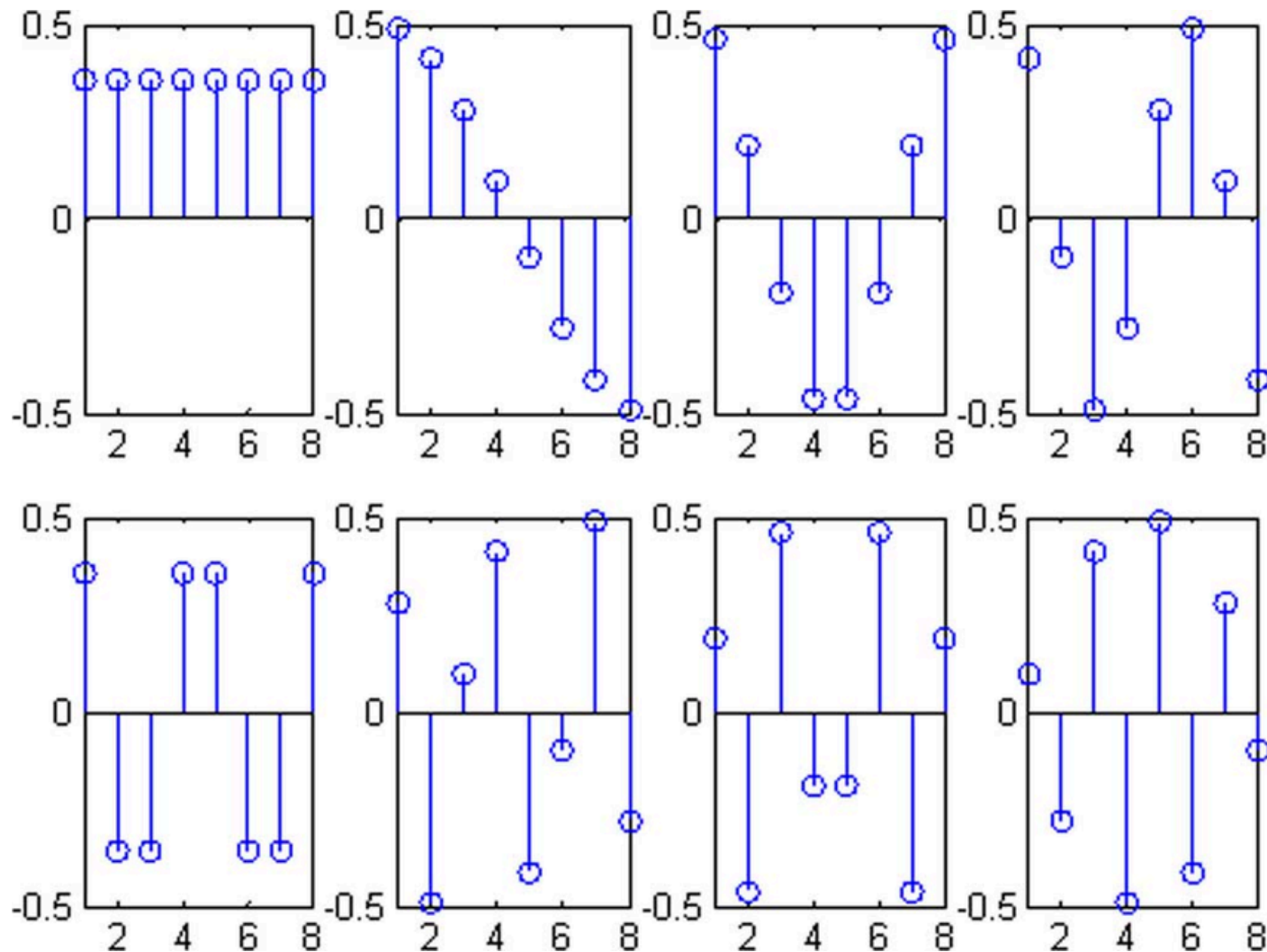
La DCT



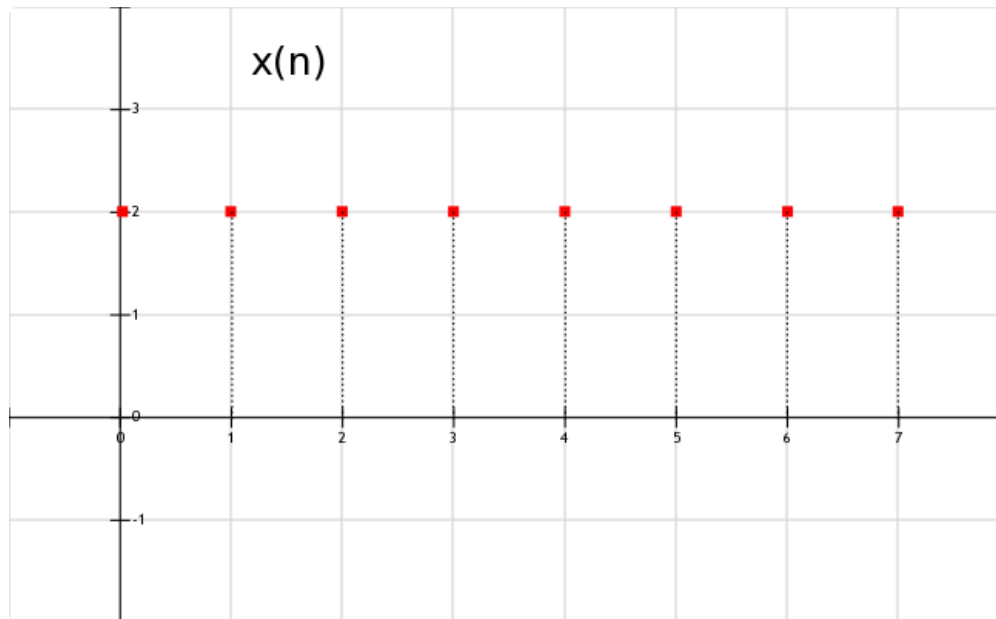
■ Ejemplo bases de la DCT con 8 funciones:



■ Las mismas bases en discreto:

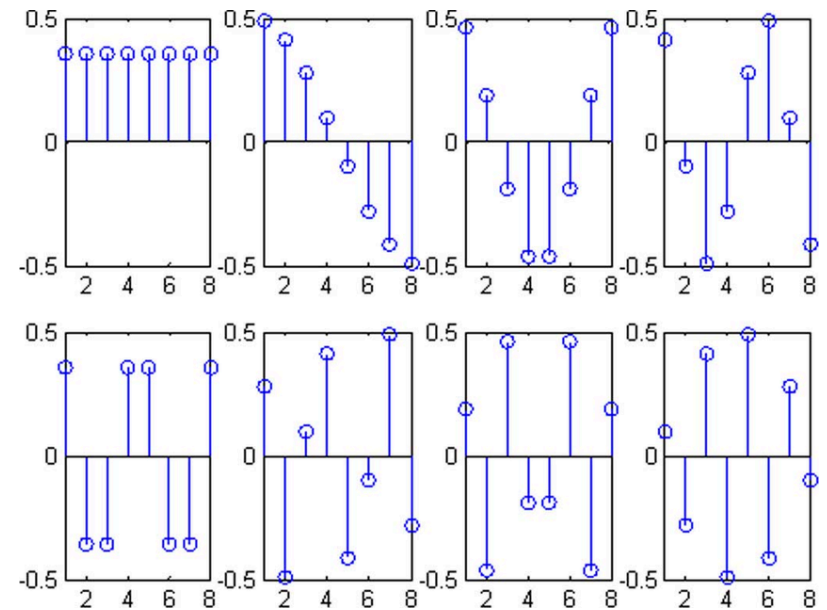


■ Ejemplo 1:

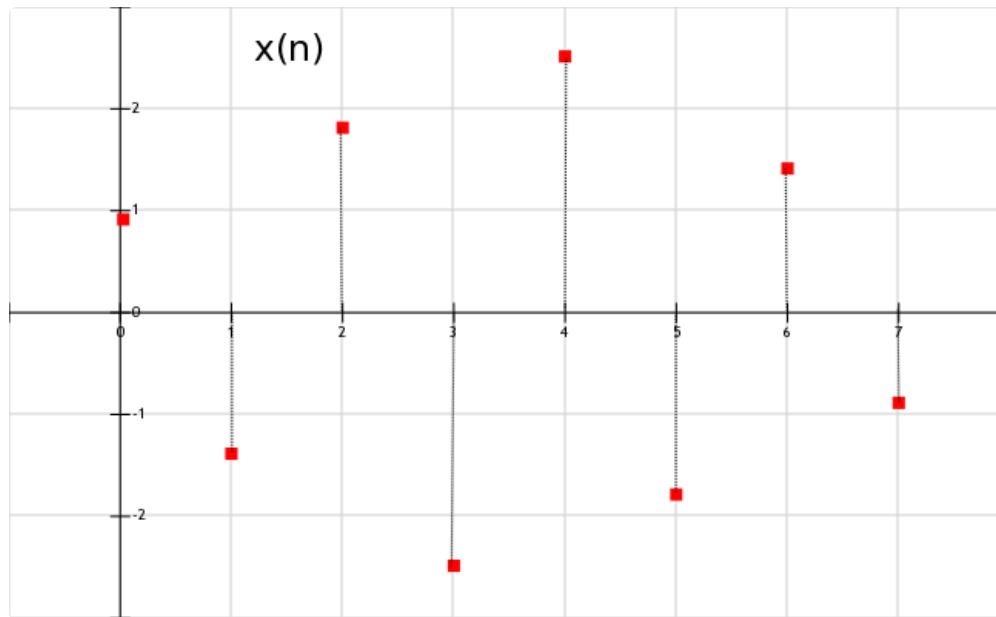


$$a_0 = 4\sqrt{2}$$

$$a_k = 0 \quad \forall k \neq 0$$

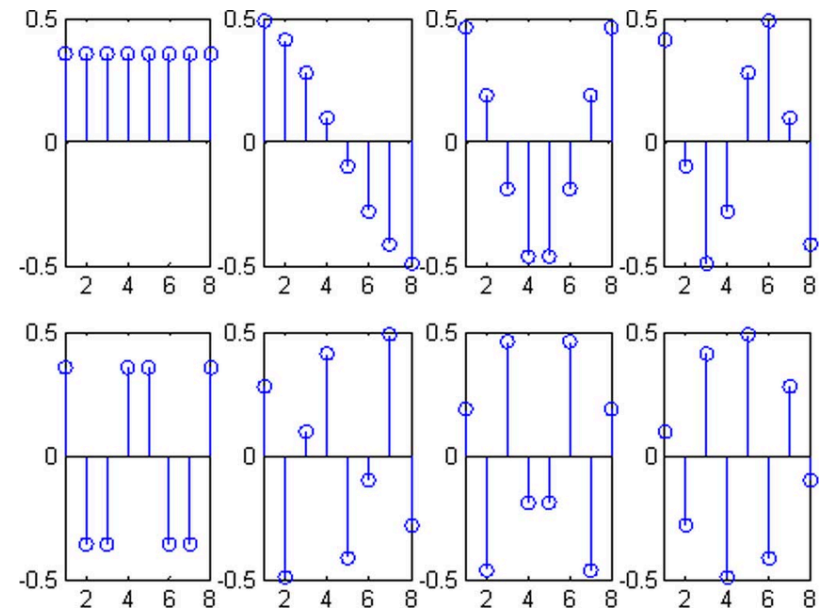


■ Ejemplo 2:

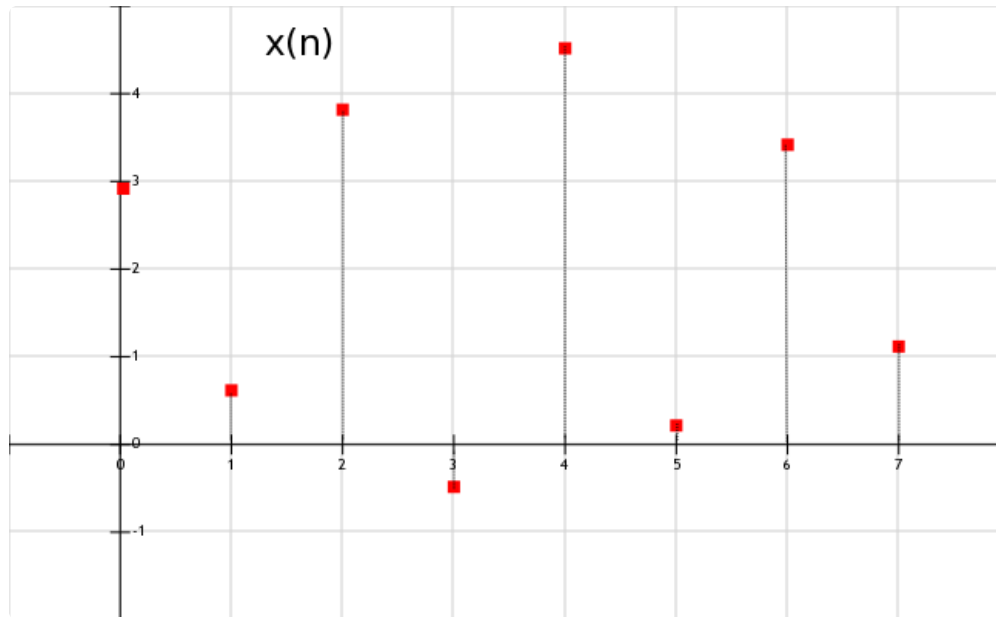


$$a_7 \approx 5$$

$$a_k = 0 \quad \forall k \neq 7$$



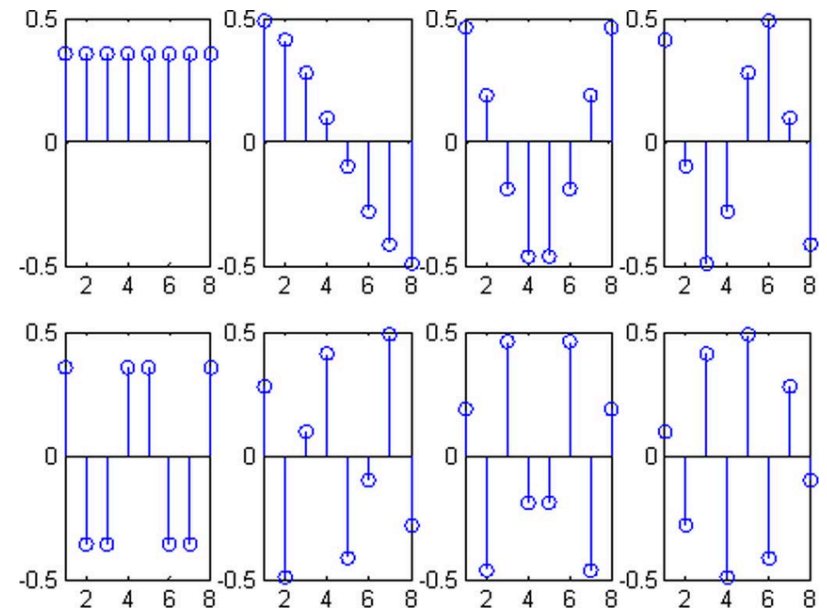
■ Ejemplo 3:



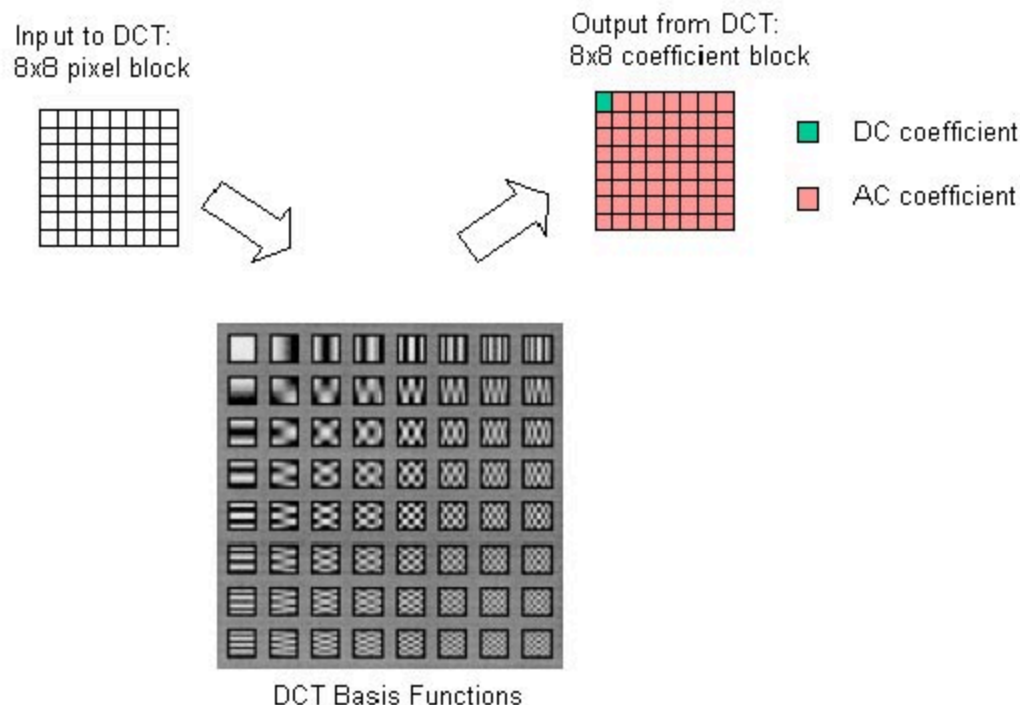
$$a_1 \approx 4\sqrt{2}$$

$$a_7 \approx 5$$

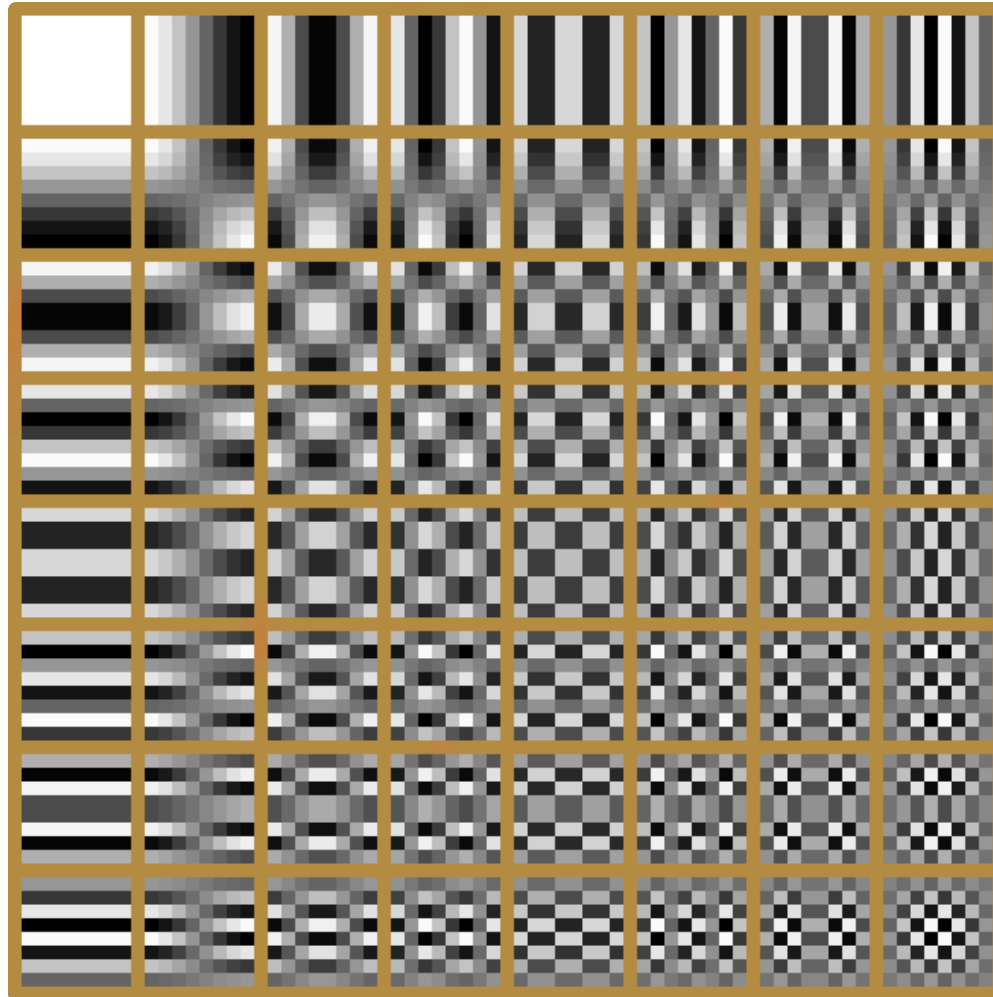
$$a_k = 0 \quad \forall k \neq 1, 7$$



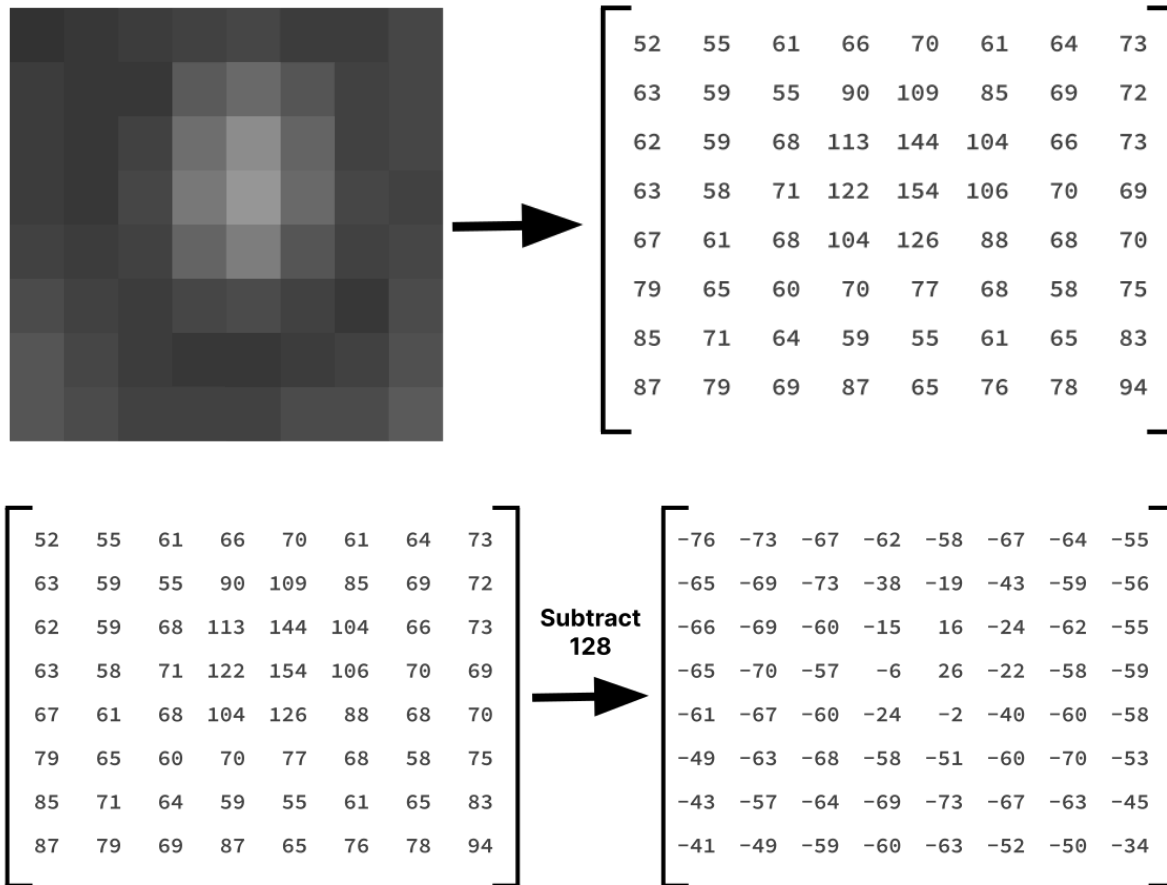
- Para cada uno de los bloques de 8x8 píxeles se aplica la DCT.
 - Más concretamente, se usa la DCT tipo-II en 2D (Discrete Cosine Transform type-II).
 - La DCT de un bloque de 8x8 píxeles (dominio espacial) es otro bloque de 8x8 píxeles en el dominio de la frecuencia.



- La frecuencia de las componentes en la DCT 2D crece desde la esquina superior izquierda hacia abajo y hacia la derecha.



- Antes de aplicar la DCT, los valores de píxeles (0-255) se centran en el rango $[-128, 127]$ restando 128 a cada uno.



- La DCT 2D calcula cada coeficiente como:

$$S_{u,v} = \frac{1}{4} C_u C_v \sum_{x=0}^7 \sum_{y=0}^7 s_{x,y} \cos \left(\frac{\pi(2x+1)u}{16} \right) \cos \left(\frac{\pi(2y+1)v}{16} \right)$$

donde:

- $s_{x,y} \in \mathbb{R}$ es el elemento de la fila x , columna y de la matriz s , con $0 \leq x \leq 7, 0 \leq y \leq 7$
- $S_{u,v} \in \mathbb{R}$ es el elemento de la fila u , columna v de la matriz $S \in \mathbb{R}^{8 \times 8}$ de coeficientes de la DCT, con $0 \leq u \leq 7, 0 \leq v \leq 7$
- C_u y C_v se definen como:

$$C_u, C_v = \begin{cases} 1/\sqrt{2}, & u, v = 0 \\ 1, & \text{en otro caso} \end{cases}$$

- Existe otra forma alternativa de obtener la DCT que reduce los cálculos a una multiplicación de matrices. La expresión sería:

$$S = U \cdot s \cdot U^T$$

- Donde S es la matriz con los valores de la DCT de s , y U es una matriz ortonormal con los siguientes valores:

$$U = \frac{1}{2} \begin{bmatrix} \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \\ \cos \frac{\pi}{16} & \cos \frac{3\pi}{16} & \cos \frac{5\pi}{16} & \cos \frac{7\pi}{16} & \cos \frac{9\pi}{16} & \cos \frac{11\pi}{16} & \cos \frac{13\pi}{16} & \cos \frac{15\pi}{16} \\ \cos \frac{2\pi}{16} & \cos \frac{6\pi}{16} & \cos \frac{10\pi}{16} & \cos \frac{14\pi}{16} & \cos \frac{18\pi}{16} & \cos \frac{22\pi}{16} & \cos \frac{26\pi}{16} & \cos \frac{30\pi}{16} \\ \cos \frac{3\pi}{16} & \cos \frac{9\pi}{16} & \cos \frac{15\pi}{16} & \cos \frac{21\pi}{16} & \cos \frac{27\pi}{16} & \cos \frac{33\pi}{16} & \cos \frac{39\pi}{16} & \cos \frac{45\pi}{16} \\ \cos \frac{4\pi}{16} & \cos \frac{12\pi}{16} & \cos \frac{20\pi}{16} & \cos \frac{28\pi}{16} & \cos \frac{36\pi}{16} & \cos \frac{44\pi}{16} & \cos \frac{52\pi}{16} & \cos \frac{60\pi}{16} \\ \cos \frac{5\pi}{16} & \cos \frac{15\pi}{16} & \cos \frac{25\pi}{16} & \cos \frac{35\pi}{16} & \cos \frac{45\pi}{16} & \cos \frac{55\pi}{16} & \cos \frac{65\pi}{16} & \cos \frac{75\pi}{16} \\ \cos \frac{6\pi}{16} & \cos \frac{18\pi}{16} & \cos \frac{30\pi}{16} & \cos \frac{42\pi}{16} & \cos \frac{54\pi}{16} & \cos \frac{66\pi}{16} & \cos \frac{78\pi}{16} & \cos \frac{90\pi}{16} \\ \cos \frac{7\pi}{16} & \cos \frac{21\pi}{16} & \cos \frac{35\pi}{16} & \cos \frac{49\pi}{16} & \cos \frac{63\pi}{16} & \cos \frac{77\pi}{16} & \cos \frac{91\pi}{16} & \cos \frac{105\pi}{16} \end{bmatrix}$$

■ Ejemplo de cálculo de la DCT:

$$s = \begin{bmatrix} 100 & 100 & 100 & 100 & 100 & 100 & 100 & 100 \\ 100 & 100 & 100 & 100 & 100 & 100 & 100 & 100 \\ 100 & 100 & 100 & 100 & 100 & 100 & 100 & 100 \\ 100 & 100 & 100 & 100 & 100 & 100 & 100 & 100 \\ 100 & 100 & 100 & 100 & 100 & 100 & 100 & 100 \\ 100 & 100 & 100 & 100 & 100 & 100 & 100 & 100 \\ 100 & 100 & 100 & 100 & 100 & 100 & 100 & 100 \\ 100 & 100 & 100 & 100 & 100 & 100 & 100 & 100 \end{bmatrix}$$

■ La DCT resultante sería:

$$S = \begin{bmatrix} 800 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

■ Otro ejemplo de cálculo de la DCT:

$$s = \begin{bmatrix} 51 & 52 & 51 & 50 & 50 & 52 & 50 & 52 \\ 51 & 52 & 51 & 51 & 50 & 52 & 52 & 51 \\ 50 & 50 & 51 & 52 & 52 & 51 & 51 & 51 \\ 51 & 50 & 50 & 50 & 52 & 50 & 50 & 51 \\ 51 & 50 & 50 & 51 & 50 & 50 & 51 & 50 \\ 50 & 51 & 52 & 52 & 51 & 50 & 50 & 50 \\ 51 & 52 & 51 & 50 & 52 & 50 & 52 & 50 \\ 50 & 51 & 52 & 52 & 50 & 51 & 52 & 51 \end{bmatrix}$$

■ La DCT resultante sería:

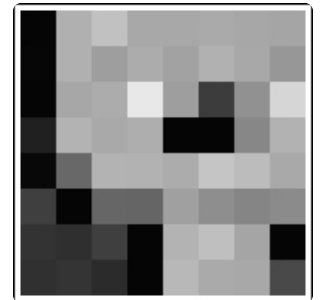
$$S = \begin{bmatrix} 407 & 0.058 & -0.518 & -0.592 & -0.5 & 0.118 & -0.597 & 0.086 \\ 0.352 & -0.654 & 1.019 & 0.818 & 0.179 & -1.074 & 1.190 & -1.194 \\ 1.904 & -0.116 & 1 & -0.598 & -2.174 & -0.352 & 0.293 & -1.006 \\ -0.661 & 1.350 & 0.689 & -0.055 & -0.425 & -0.599 & 0.254 & -0.412 \\ -1 & -0.335 & 1.171 & 0.102 & 0.5 & -0.020 & 0.868 & -0.502 \\ -0.229 & 0.162 & 0.115 & 0.711 & 0.956 & -1.902 & -0.108 & 1.454 \\ 0.023 & -0.173 & -1.707 & -1.529 & 0.630 & 0.109 & 1 & -0.603 \\ -0.110 & -0.383 & 0.105 & 0.470 & 0.005 & 0.568 & -0.470 & 0.111 \end{bmatrix}$$

- Para el bloque de antes, sacado de la imagen del ejemplo (con los valores en el rango -128,127):

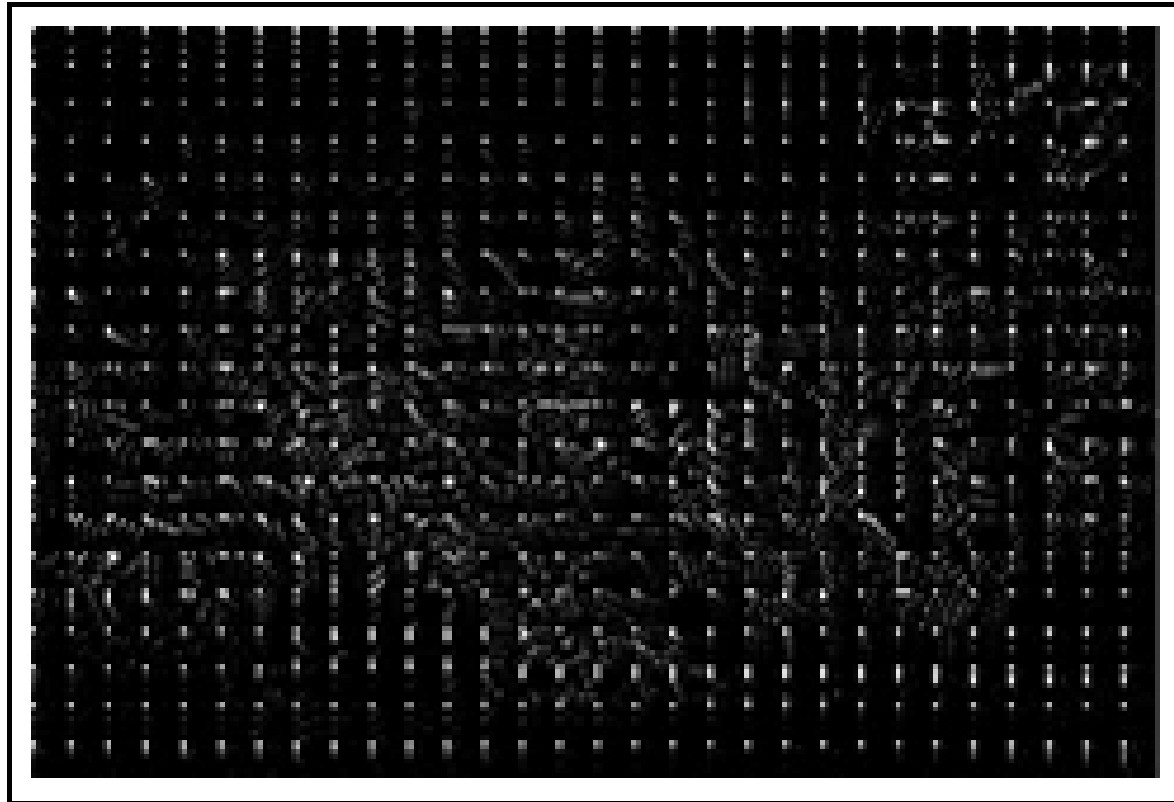
$$s = \begin{bmatrix} -122 & 49 & 66 & 41 & 41 & 43 & 40 & 38 \\ -121 & 49 & 31 & 45 & 35 & 50 & 41 & 24 \\ -122 & 40 & 45 & 105 & 31 & -66 & 18 & 87 \\ -94 & 52 & 42 & 47 & -122 & -122 & 8 & 51 \\ -119 & -23 & 53 & 51 & 45 & 70 & 61 & 42 \\ -64 & -122 & -25 & -26 & 33 & 15 & 6 & 12 \\ -76 & -80 & -64 & -122 & 53 & 64 & 38 & -122 \\ -78 & -74 & -84 & -122 & 57 & 43 & 41 & -53 \end{bmatrix}$$

- La DCT resultante sería:

$$S = \begin{bmatrix} -27.500 & -213.468 & -149.608 & -95.281 & -103.750 & -46.946 & -58.717 & 27.226 \\ 168.229 & 51.611 & -21.544 & -239.520 & -8.238 & -24.495 & -52.657 & -96.621 \\ -27.198 & -31.236 & -32.278 & 173.389 & -51.141 & -56.942 & 4.002 & 49.143 \\ 30.184 & -43.070 & -50.473 & 67.134 & -14.115 & 11.139 & 71.010 & 18.039 \\ 19.500 & 8.460 & 33.589 & -53.113 & -36.750 & 2.918 & -5.795 & -18.387 \\ -70.593 & 66.878 & 47.441 & -32.614 & -8.195 & 18.132 & -22.994 & 6.631 \\ 12.078 & -19.127 & 6.252 & -55.157 & 85.586 & -0.603 & 8.028 & 11.212 \\ 71.152 & -38.373 & -75.924 & 29.294 & -16.451 & -23.436 & -4.213 & 15.624 \end{bmatrix}$$



- Si hacemos la DCT de todos los bloques de la imagen de ejemplo del principio y los coloreamos según el valor absoluto de los coeficientes, obtenemos:



- El resultado de la DCT ocupa temporalmente más espacio que la matriz original de intensidades de los píxeles.
 - En la práctica, cada valor del coeficiente de la salida de la DCT se representa en 16 bits (cuando la entrada son valores en 8 bits).
 - Este incremento temporal de la imagen podría llegar a suponer un problema de memoria en la fase de compresión.
 - Una estrategia posible es evitar almacenar toda la imagen transformada.

- Para eliminar la información de las altas frecuencias, se usa la matriz de cuantificación.
- La matriz de DCT se divide por la matriz de cuantificación, y los valores se redondean al entero mas cercano.

$$K = \lfloor \frac{S}{Q} \rfloor$$

- La idea es conseguir muchos ceros en la matriz resultante, ya que estos después se codifican con muy pocos bits.
- Por ello la matriz de cuantificación tiene valores mas grandes en los valores correspondientes a las frecuencias mas altas.

■ Ejemplo de matriz de cuantificación:

16	11	10	16	24	40	51	61
12	12	14	19	26	58	60	55
14	13	16	24	40	57	69	56
14	17	22	29	51	87	80	62
18	22	37	56	68	109	103	77
24	35	55	64	81	104	113	92
49	64	78	87	103	121	120	101
72	92	95	98	112	100	103	99

- La matriz de cuantificación se guarda en el fichero JPEG para la posterior decodificación.

■ Resultado de hacer $K = \lfloor \frac{s}{Q} \rfloor$.

-26	-3	-6	2	2	-1	0	0
0	-2	-4	1	1	0	0	0
-3	1	5	-1	-1	0	0	0
-3	1	2	-1	0	0	0	0
1	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

- La cuantificación comprime de forma irreversible (con pérdidas) los valores transformados (salida de la DCT):
 - Hay una operación de redondeo después de cuantificar.
 - Los valores próximos a cero son realmente truncados a cero.
 - Obviamente, el resultado es irreversible.
- Se reduce la información correspondiente a las componentes de alta frecuencia del bloque.
 - Normalmente los valores truncados son los coeficientes AC correspondientes a las frecuencias más altas
 - Se asumen imágenes con transiciones suaves donde la información reside en el coeficiente DC y los coeficientes AC de menor frecuencia

Funcionamiento Cuantificación

Codificación JPEG: Ejemplo real (Quant)

40	44	47	40	40	55	79	75
44	52	40	47	40	48	67	79
52	55	36	67	63	62	52	72
68	45	56	60	52	55	36	60
62	48	56	48	40	36	47	62
47	67	40	55	55	40	36	62
36	56	23	67	62	44	49	47
48	55	36	55	52	47	47	36

Bloque de muestras (pixels)

DCT

411	-18	14	-8	24	-10	-14	-18
20	-34	27	-9	-11	11	14	7
-11	-23	-1	5	-19	4	-20	-2
-8	-5	14	-14	-8	-3	-3	9
-3	9	7	2	-10	17	18	16
3	-2	-17	8	7	-3	1	-8
8	1	-2	3	-2	-7	-1	-2
1	-8	-4	2	2	3	-7	2

Bloque de muestras transformadas

Quant

39	49	38	47	45	55	70	76
50	49	38	46	45	57	65	72
54	46	43	55	54	60	53	60
58	47	50	59	55	57	43	55
64	52	51	54	45	48	41	60
56	52	51	55	44	45	42	55
43	50	47	58	51	49	43	42
42	50	40	53	50	51	48	39

Bloque recuperado de muestras

IDCT

102	-2	1	0	1	0	0	-1
2	-4	2	0	0	0	0	0
-1	-2	0	0	-1	0	-1	0
0	0	1	-1	0	0	0	0
0	0	0	0	0	0	0	0
0	0	-1	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

Bloque de muestras cuantizadas

- *Nota:* en la imagen, antes de hacer la IDCT habría que multiplicar el bloque de muestras cuantizadas por la matriz de cuantificación.

- La relación de compresión (relación entre el tamaño de la imagen comprimida y la original) puede variarse escalando la matriz de cuantificación.
 - Para escalar la matriz de cuantificación, se multiplica cada elemento por un factor de escala.
 - Compresiones 10:1 dan como resultado imágenes indistinguibles de la original por el ojo humano.
 - Compresiones 100:1 manifiestan artefactos y distorsiones claramente visibles.
- ¡Atención!
 - La calidad final de la imagen comprimida JPEG depende del ratio de compresión pero también de la calidad de la imagen original.

- Para el bloque de ejemplo, usando la matriz de cuantificación Q del ejemplo anterior, se obtendría:

$$K = \begin{bmatrix} -2 & -19 & -15 & -6 & -4 & -1 & -1 & 0 \\ 14 & 4 & -2 & -13 & 0 & 0 & -1 & -2 \\ -2 & -2 & -2 & 7 & -1 & -1 & 0 & 1 \\ 2 & -3 & -2 & 2 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & -1 & -1 & 0 & 0 & 0 \\ -3 & 2 & 1 & -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 & 1 & 0 & 0 & 0 \\ 1 & 0 & -1 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

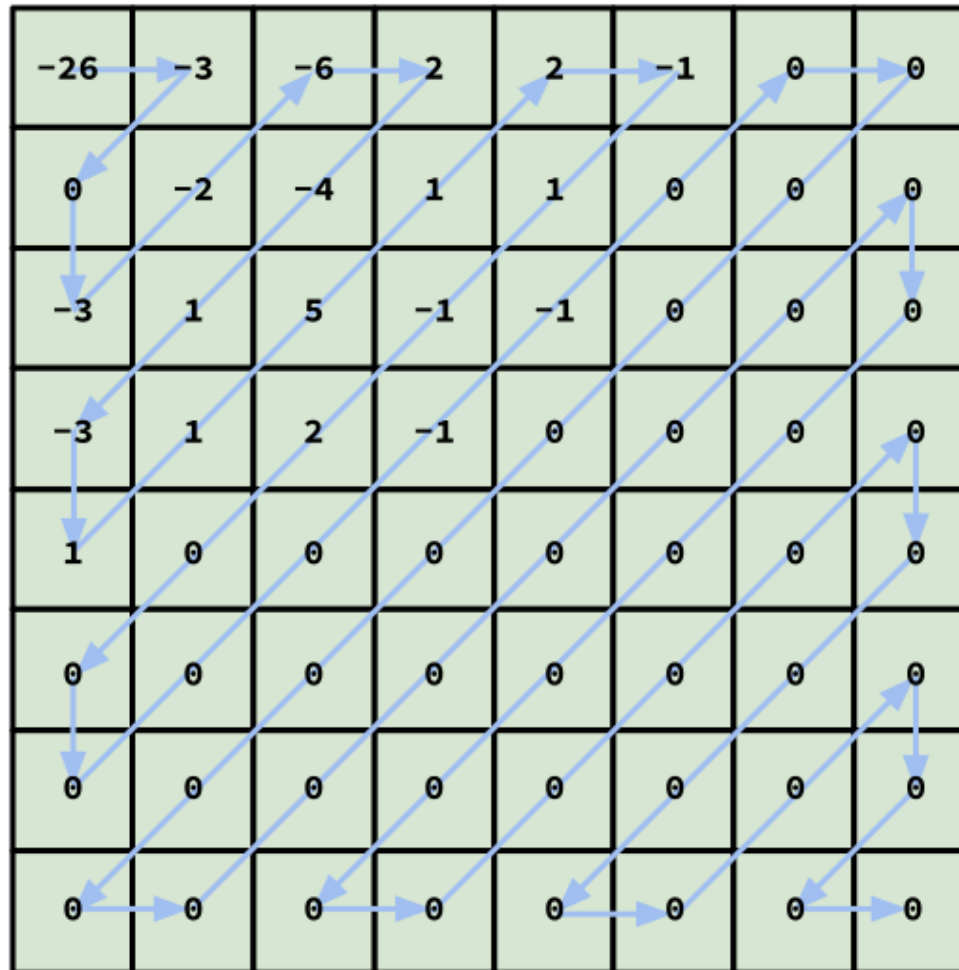
Valores más pequeños en la esquina superior izquierda (bajas frecuencias) y valores mucho mayores en el resto (altas frecuencias).

- La última fase del proceso de compresión consiste en codificar los valores de la matriz de la DCT cuantificada.
- Se usa una codificación entrópica que no pierde información.
 - Entropía: cantidad de información media de los valores.
 - Los valores menos frecuentes aportan más información que los más frecuentes.
 - Ejemplo:
 - "XX codificación entrópica XX XX tipo XX compresión XX datos sin pérdida."
 - "La XX XX es un tipo de XX de datos sin XX."

- Para codificar, los valores de la matriz de la DCT cuantificada se reordenan en zig-zag, de forma que los valores de las frecuencias bajas van primero.
- La idea es que queden muchos ceros seguidos al final. Esto hará que se pueda comprimir mucho la información usando una codificación *run-length*. RLE

Funcionamiento

Codificación entrópica



-26 -3 0 -3 -2 -6 2 -4 1 -3 1 1 5 1 2 -1 1 -1 2 0 0 0 0 0 0 -1 -1 0 0 0 0 0 0 0 . . .

- Codificación de ráfaga (RLE, Run-Length Encoding) :
 - Busca coeficientes AC no nulos, determina el número de ceros anteriores y genera dos símbolos de 1 byte cada uno:
 - *Símbolo 1*: (RUNLENGTH, SIZE)
 - *Símbolo 2*: (AMPLITUDE)

- *RUNLENGTH* es el número de ceros de la secuencia anteriores al coeficiente no nulo.
 - Es un campo de 4 bits, por lo que sólo permite valores [0,15].
 - Las secuencias mayores de 16 ceros son divididas.
 - Una ráfaga de 16 ceros se codifica con los símbolos (15, 0) (0).
- *SIZE*: es el número de bits usados para codificar *AMPLITUDE*.
- *AMPLITUDE*: es el valor en sí del coeficiente.
- Si hay ceros hasta el final de la secuencia, se codifica con el símbolo (0,0).

Funcionamiento

Codificación entrópica

SIZE	Valor de amplitud
1	-1, 1
2	-3, -2, 2, 3
3	-7..-4, 4..7
4	-15..-8, 8..15
5	-31..-16, 16..31
6	-63..-32, 32..63
7	-127..-64, 64..127
8	-255..-128, 128..255
9	-511..-256, 256..511
10	-1 023..-512, 512..1 023

- Para calcular los bits correspondientes a cada amplitud, se usa el número de bits definido en SIZE de la siguiente forma:
 - Si el valor es positivo, se codifica el valor en binario:
 - P.Ej. 8 tendría SIZE=4, y sus bits serían 1000
 - Si el valor es negativo, se le resta 1 y se hace el complemento a 2:
 - P.Ej. -8: Se resta 1: -9. -9 en complemento a 2:
 - Valor absoluto en binario 1001
 - Se invierte 0110
 - Se suma 1: 0111

■ Ejemplo con los valores del bloque de ejemplo:

$$K = \begin{bmatrix} -2 & -19 & -15 & -6 & -4 & -1 & -1 & 0 \\ 14 & 4 & -2 & -13 & 0 & 0 & -1 & -2 \\ -2 & -2 & -2 & 7 & -1 & -1 & 0 & 1 \\ 2 & -3 & -2 & 2 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & -1 & -1 & 0 & 0 & 0 \\ -3 & 2 & 1 & -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 & 1 & 0 & 0 & 0 \\ 1 & 0 & -1 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

■ Siguiendo el recorrido en zig-zag, se obtendría la siguiente secuencia:

-2, -19, 14, -2, 4, -15, -6, -2, -2, 2, 1, -3, -2, -13, -4,
-1, 0, 7, -2, 0, -3, 0, 2, 1, 2, -1, 0, -1, 0, -1, -1, 0, -1,
1, 0, 1, 0, 0, -1, -1, 0, 0, -2, 1, 1, 0, 0, -1, -1, 0, 1, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0

Funcionamiento

Codificación entrópica

-2, -19, 14, -2, 4, -15, -6, -2, -2, 2, 1, -3, -2, -13, -4,
-1, 0, 7, -2, 0, -3, 0, 2, 1, 2, -1, 0, -1, 0, -1, -1, 0, -1,
1, 0, 1, 0, 0, -1, -1, 0, 0, -2, 1, 1, 0, 0, -1, -1, 0, 1, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0

- Se observa una larga secuencia de ceros al final.
- Si los coeficientes de cuantificación fuesen mayores, se hubiese generado una secuencia de ceros todavía más larga.
- El primer elemento de la secuencia es la componente de DC, mientras que las restantes son componentes de AC ordenadas de menor a mayor frecuencia.

-2, -19, 14, -2, 4, -15, -6, -2, -2, 2, 1, -3, -2, -13, -4, -1, 0, 7, -2, 0, -3, 0,
2, 1, 2, -1, 0, -1, 0, -1, -1, 0, -1, 1, 0, 1, 0, 0, -1, -1, 0, 0, -2, 1, 1, 0, 0, -1,
-1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0

■ Para codificar, usamos RLE:

DC: (2, -2); AC: (0, 5)(-19); (0, 4)(14); ... (0, 1)(-1); (1, 3)(7); ... (1, 1)(1);
(0, 0)

- Tras el paso del algoritmo RLE, se aplica Huffman para codificar los valores AC con el mínimo número de bits posibles.
- Para ello se obtiene un código de unas tablas Huffman predefinidas para los valores (RUNLENGTH, SIZE) y se concatena el valor de la amplitud.
- Algunos ejemplos de códigos Huffman de la tabla estandar:

(RUNLENGTH, SIZE)	Código Huffman
(0, 0)	1010
(0, 2)	01
(1, 1)	1100
(15, 0)	111111111001

- No hay códigos definidos para todas las posibles combinaciones de (RUNLENGTH, SIZE).
 - La mayoría de las combinaciones con RUNLENGTH alto y SIZE grande nunca se dan en imágenes naturales (la energía se concentra en las frecuencias bajas).
 - El estándar JPEG permite que el codificador puede generar sus propias tablas Huffman (se guardan en el archivo final).

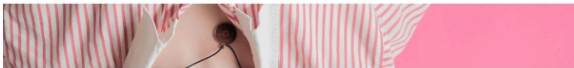
- La componente DC se codifica de forma especial.
 - Se codifica la diferencia con la componente DC del bloque anterior.
 - Se calcula el SIZE igual para los componentes AC.
 - Los valores de SIZE tienen un código en otra tabla Huffman.
 - El valor final concatena ese código con el valor de la amplitud.

- Tabla con los códigos Huffman para los valores de SIZE de la componente DC:

SIZE	Código Huffman	SIZE	Código Huffman
0	00	6	1110
1	010	7	11110
2	011	8	111110
3	100	9	1111110
4	101	10	11111110
5	110		

- Hay dos tipos de codificación según el orden en que se codifican los coeficientes:
 - Secuencial:
 - Se corresponde con el esquema anterior en el que se realiza una codificación independiente para cada bloque de 8x8 píxeles.

Baseline JPEG - Loads from top-to-bottom



■ Progresiva:

- Se codifica primero la posición (0,0) para N bloques de 8x8 píxeles.
- A continuación, se codifican las posiciones (0,1) y (1,0) de los N bloques de 8x8 píxeles y así sucesivamente.
- Es decir, sean B_i los bloques con $i = 1, \dots, N$
 - Se codifica primero $B_1(0, 0), B_2(0, 0), \dots, B_N(0, 0)$.
 - Y luego: $B_1(0, 1), B_2(0, 1), \dots, B_N(0, 1), B_1(1, 0), B_2(1, 0), \dots$
- Permite utilizar una variante de Huffman que proporciona resultados de compresión mejores.
- Mayor tasa de compresión
- Normalmente necesita más memoria y CPU

Progressive JPEG - Loads from low-quality to high-quality



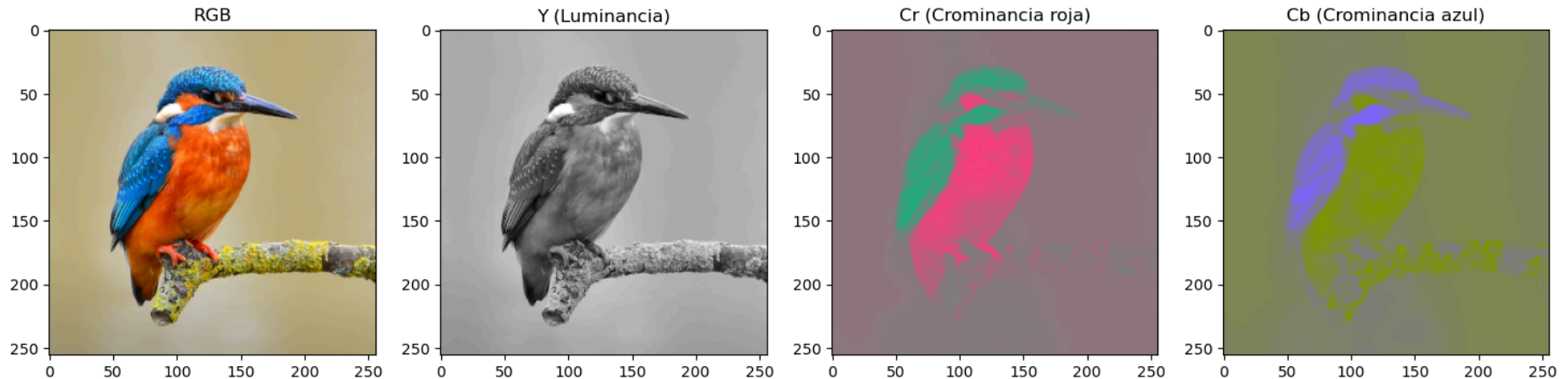
- Partimos de la siguiente imagen:



Funcionamiento

Ejemplo completo

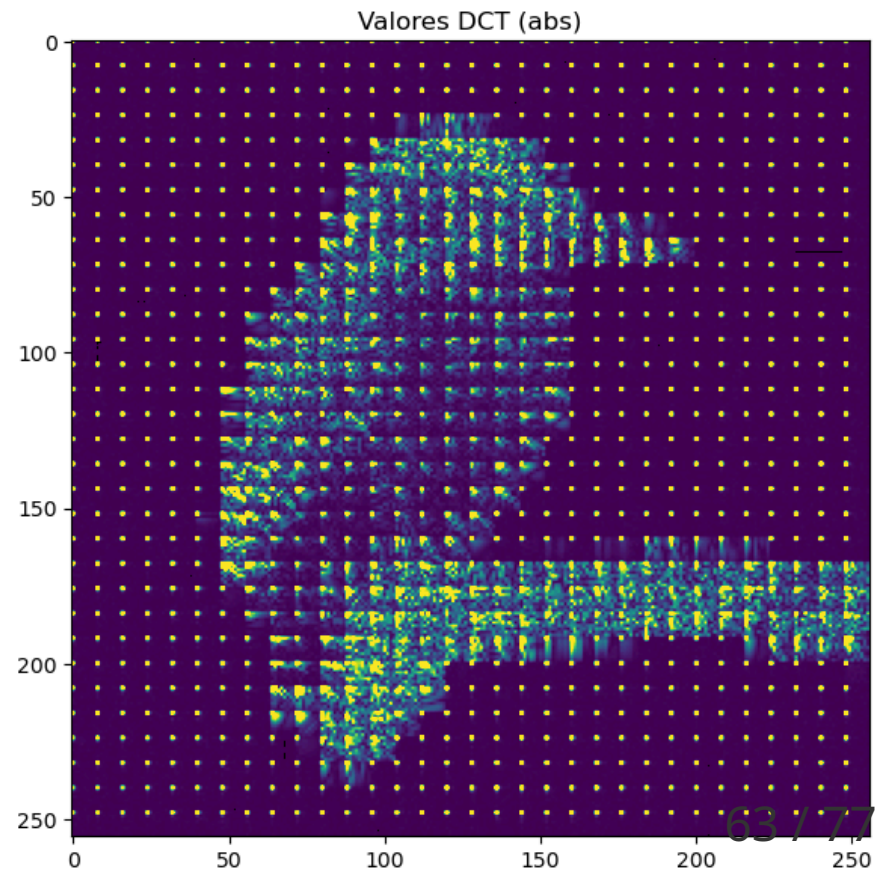
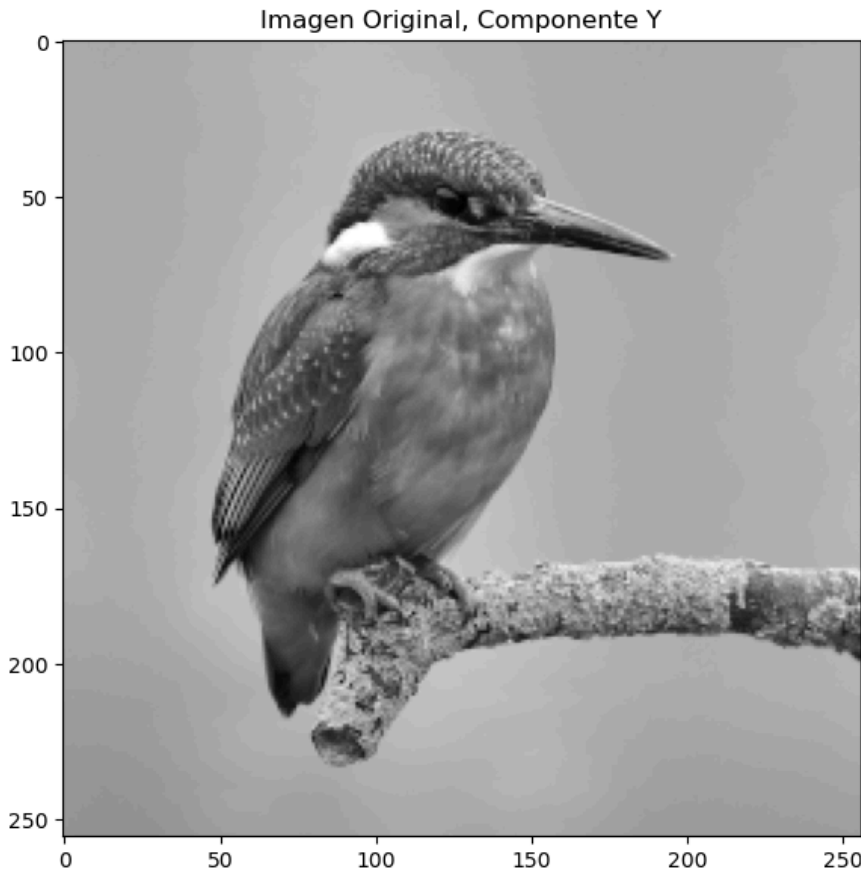
■ Separamos color de luminancia:



Funcionamiento

Ejemplo completo

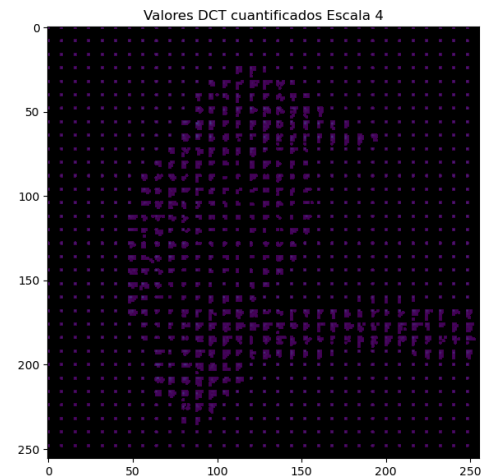
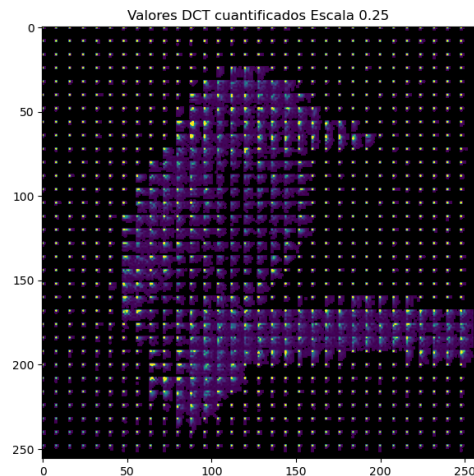
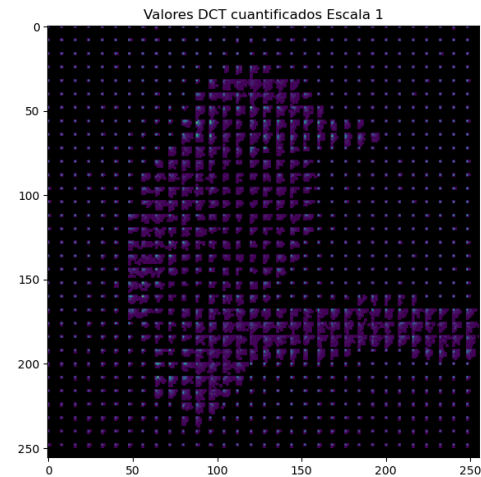
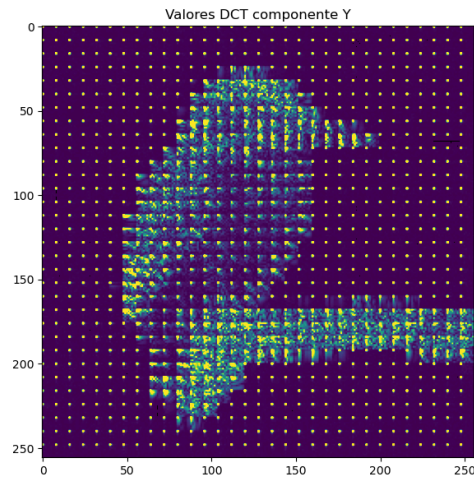
- Hacemos un downsampling de crominancia.
- Dividimos en bloques de 8x8 píxeles.
- Aplicamos la DCT a cada bloque.



Funcionamiento

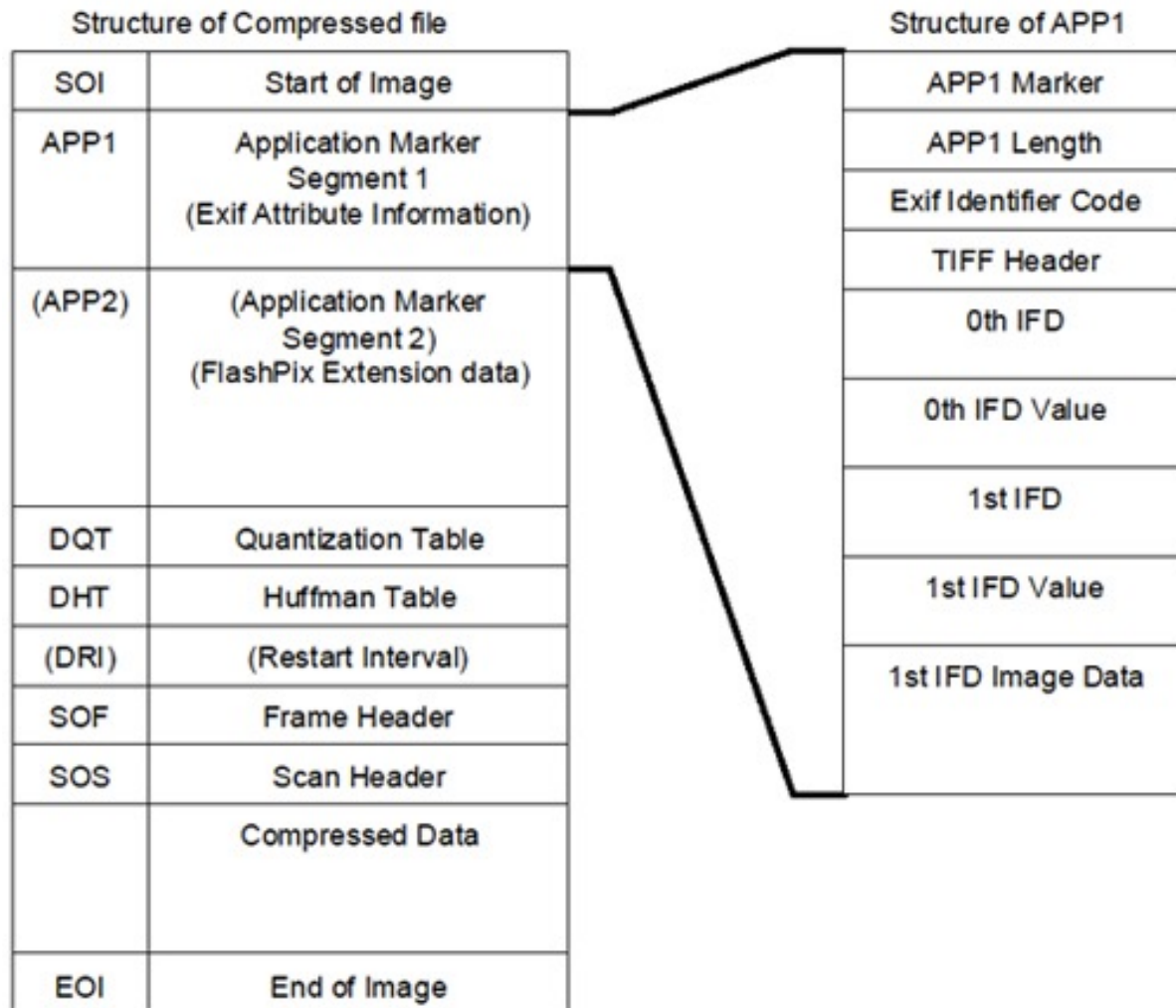
Ejemplo completo

- Dependiendo del escalado de la matriz de cuantificación, eliminaremos más o menos información:



- JPEG es una especificación para comprimir imágenes, no un formato de fichero.
- Los formatos de fichero que almacenan datos siguiendo la especificación JPEG son dos: JPEG File Interchange Format (JFIF) y Exchangeable image file format (EXIF).
- Ambos formatos permiten añadir metadatos y funcionalidad extra.
- La extensión del fichero es la misma, .jpeg.

■ Estructura EXIF:



Marcador	Nombre	Descripción
SOI	<i>Start of Image</i>	Marca el inicio del archivo JPEG. Siempre es <code>FFD8</code> .
APP1	<i>Application Marker Segment 1</i>	Aquí se guarda la información <i>EXIF</i> (metadatos como cámara, exposición, orientación, etc.).
(APP2)	<i>Application Marker Segment 2</i>	Puede contener información extra (como FlashPix, menos común).
DQT	<i>Define Quantization Table</i>	Tablas usadas para cuantizar los coeficientes DCT.

Marcador	Nombre	Descripción
DHT	<i>Define Huffman Table</i>	Tablas Huffman usadas para la codificación.
(DRI)	<i>Define Restart Interval</i>	Opcional. Indica intervalos de reinicio en el flujo de datos.
SOF	<i>Start of Frame</i>	Contiene la información de la imagen: tamaño, número de componentes, etc.
SOS	<i>Start of Scan</i>	Indica el comienzo de los datos comprimidos de la imagen.

Marcador	Nombre	Descripción
Compressed Data		Aquí están los datos JPEG comprimidos (coeficientes DCT codificados con Huffman).
EOI	<i>End of Image</i>	Marca el final del archivo (FFD9).

■ Ejemplo de metadatos EXIF:

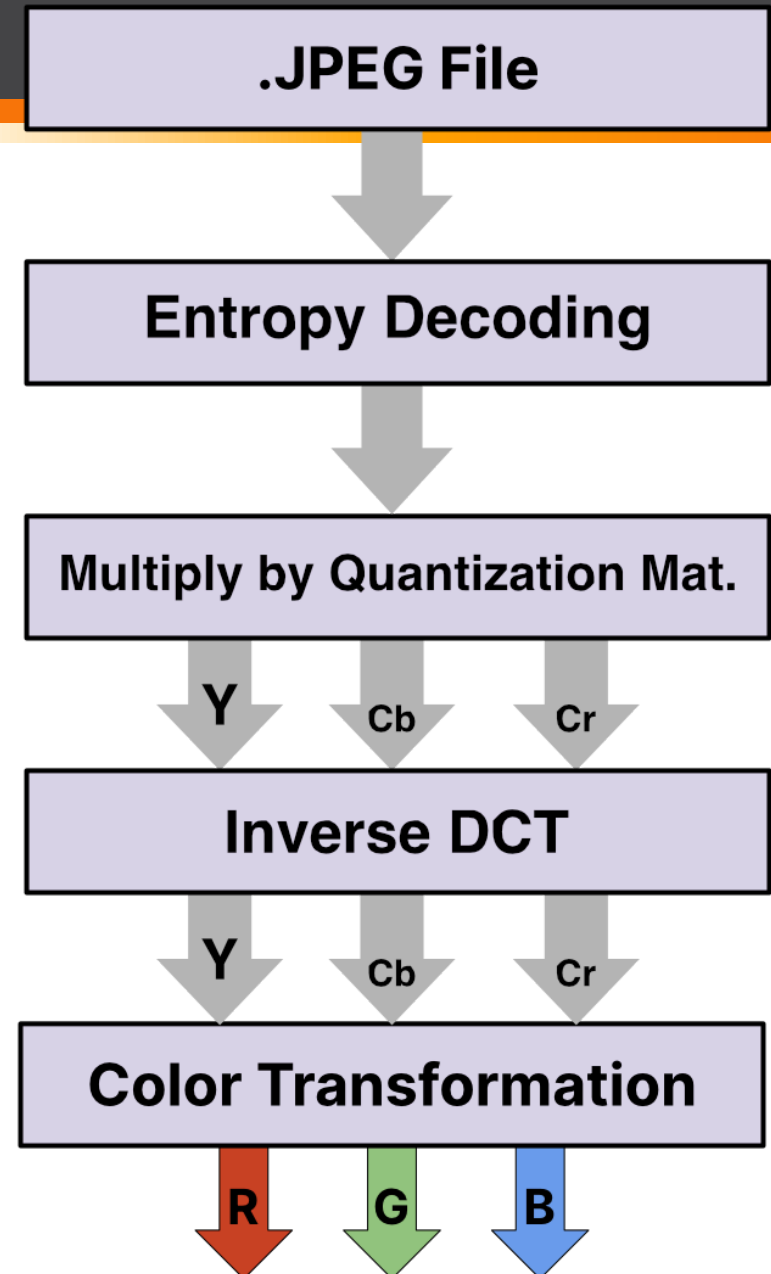
Property	Value
Origin	
Authors	
Date taken	14/12/2003 12:01
Program name	
Date acquired	
Copyright	

Image	
Image ID	
Dimensions	480 x 360
Width	480 pixels
Height	360 pixels
Horizontal resolution	180 dpi
Vertical resolution	180 dpi
Bit depth	24
Compression	
Resolution unit	2
Colour representation	sRGB
Compressed bits/pixel	5

Property	Value
Camera	
Camera maker	Canon
Camera model	Canon PowerShot S40
F-stop	f/4.9
Exposure time	1/500 sec.
ISO speed	
Exposure bias	0 step
Focal length	21 mm
Max aperture	2.970855712890625
Metering mode	Centre Weighted Average
Subject distance	
Flash mode	No flash, auto
Flash energy	
35mm focal length	

Advanced photo	
Lens maker	
Lens model	
Flash maker	
Flash model	

- La decodificación en JPEG se realiza invirtiendo los pasos del codificador.



- Tras decodificar los valores, usamos la matriz de cuantificación para recuperar los valores de la DCT (que ya no serán los originales, tras el redondeo).
- Para volver a los valores de los pixels originales, usamos la *DCT* inversa (*IDCT*). Esta operación se define en el estándar como:

$$s_{x,y} = \frac{1}{4} \sum_{u=0}^7 \sum_{v=0}^7 C_u C_v S_{u,v} \cos \left(\frac{\pi(2x+1)u}{16} \right) \cos \left(\frac{\pi(2y+1)v}{16} \right)$$

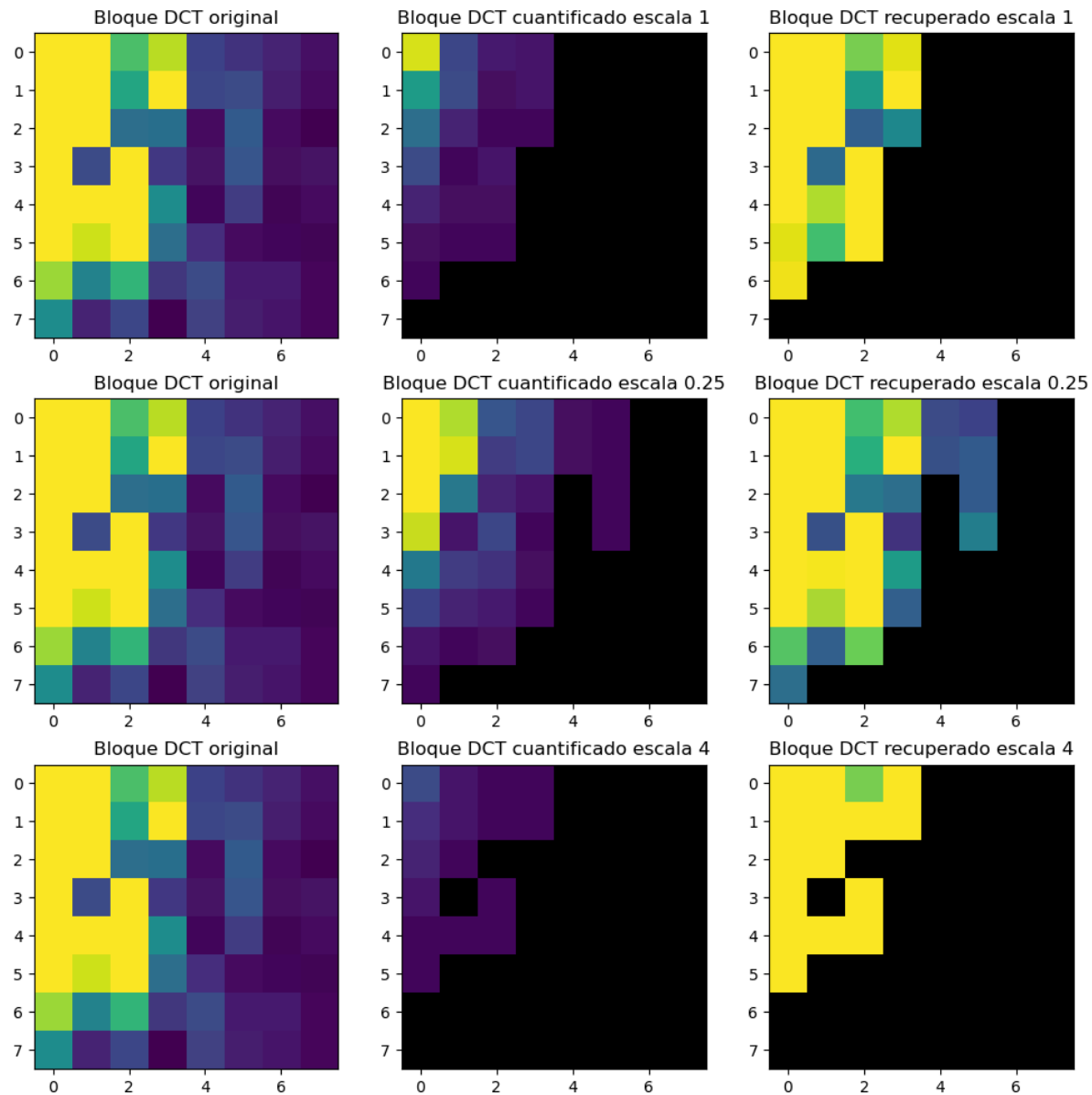
- Existe una forma alternativa de calcular la DCT inversa utilizando una simple multiplicación de matrices:

$$s = U^T \cdot S \cdot U$$

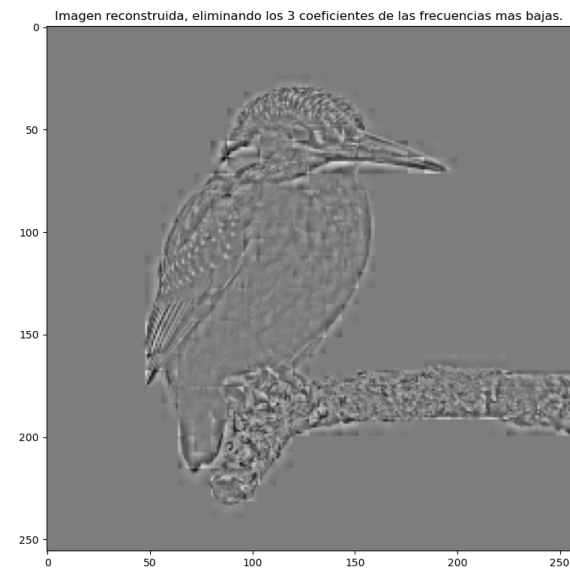
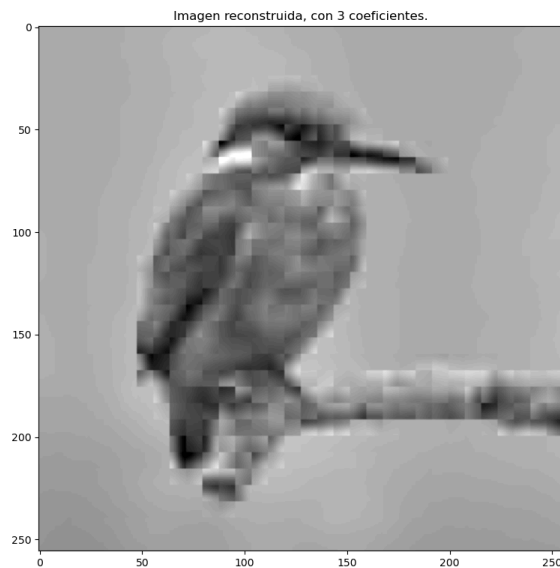
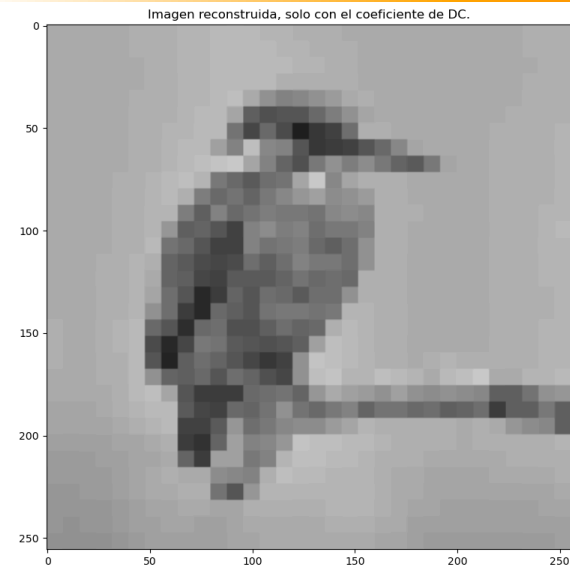
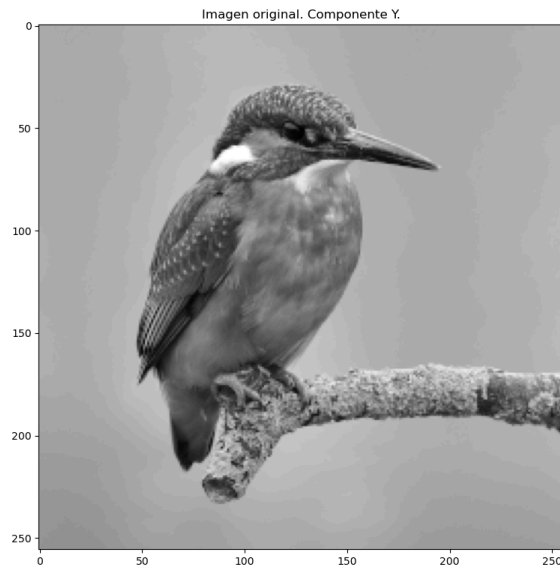
Donde U es la matriz de coeficientes de la DCT que vimos antes.

- La $IDCT$, al igual que la DCT , no elimina información, simplemente la transforma a otro dominio.
- Pero como parte de valores cuantificados, los píxeles obtenidos tras la $IDCT$ ya no serán exactamente iguales que los originales.

Funcionamiento Decodificación



Funcionamiento Decodificación



- JPEG no tiene modo sin pérdida real. (Existe uno definido, pero casi ningún programa lo soporta).
- La imagen se divide en bloques fijos de 8x8 píxeles.
 - Cada bloque se codifica de forma independiente.
 - No puede aprovechar regiones grandes con el mismo color.
 - Esto provoca transiciones visibles entre bloques.

■ *Efecto de bloque:*

- La DCT asume que el bloque es periódico, pero en realidad casi nunca es así.
- Al reconstruir (IDCT), aparecen discontinuidades en los bordes.

■ La DCT es una transformación global del bloque:

- Cada valor del píxel afecta a todos los coeficientes.
- Un pequeño error o ruido en la entrada se propaga a todo el bloque.